

Discontinuous Galerkin Methods for Convection-Dominated Problems

Jacek Schmidt, jacek14schmidt@gmail.com

February 2025

1 Introduction

Almost always, when modelling the dynamics of quantities, conservation laws must be satisfied. Whether it be the conservation of mass, energy, momentum or charge, the fundamental principle that all quantities must be accounted for generally applies: quantities cannot be created or destroyed without explanation. A direct consequence of these conservation laws is the transport equation (or continuity equation), which governs how quantities move and interact. The transport equation can be applied to a vast range of substrates, making it the governing equation underpinning many physical systems. It is typically decomposed into three main components: source, convection and diffusion. The source term represents the controlled addition or removal of substance from the system, such as a leak in a tank or the release of products in a chemical reaction. The convection term represents the bulk movement of the substance due to the flow of the surrounding medium, for example chemicals transported by a stream or particulates carried by air currents. Finally, diffusion represents the net movement of the substance down its concentration gradient as a result of random particle motion; examples include the spread of carbon monoxide in a closed room or the transfer of oxygen into the bloodstream. In many practical contexts, the contributions of the source and diffusion terms are small relative to convection. In such cases, a simplified model is adopted in which these terms are neglected, leading to convection-dominated problems.

In convection-dominated systems, the evolution of the quantity is governed entirely by the flow of the surrounding medium. The effect of the medium on the quantity is encoded in the convective flux (hereafter referred to simply as the flux), which represents both the motion of the medium and its interaction with the transported quantity. For simple fluxes, such as linear fluxes, analytical solutions to the resulting transport equation can often be derived. However, for more complex non-linear fluxes, such as Burgers' flow, analytical methods typically yield only implicit solutions, with no guarantee that these can be inverted into explicit form. In some cases, no analytical solution exists at all. In either scenario, analytical methods are insufficient to represent the system.

Many physical scenarios of interest involve rapid changes in the quantity over short spatial intervals. These shocks represent transitions between distinct states of the system. In many cases, shocks are imposed externally and correspond to physical barriers, such as boundaries between different media, doorways in crowded corridors, or abrupt changes in seabed depth due to underwater features. Alternatively, certain non-linear fluxes can give rise to shocks that form naturally, even when the initial profile is smooth. A classical example is the breaking of water waves, where wave peaks travel faster than troughs and eventually overturn. Shock profiles are characterised by very steep gradients followed by relatively flat regions, making them difficult to resolve numerically. A common simplification is to model shocks as discontinuities. Although physical shocks are not truly discontinuous, this approximation greatly simplifies both analysis and computation. However, this introduces a difficulty: many classical solutions of the transport equation assume continuity, and the presence of a discontinuity typically marks the breakdown of such solutions. Nevertheless, the evolution of systems containing distinct states remains of significant interest, and so numerical methods must be employed.

The lack of analytical solutions for convection-dominated problems with complex fluxes and initial conditions, together with the breakdown of classical solutions in the presence of discontinuities,

motivates the use of numerical approximations. A wide range of numerical methods are available for approximating solutions to the transport equation. The finite difference method (FDM) approximates derivatives using Taylor expansions on a structured grid. Its simplicity makes it easy to implement and computationally inexpensive, but it struggles to represent discontinuities in a stable and physically meaningful way. The finite volume method (FVM) instead partitions the domain into control volumes and applies conservation laws to each volume, tracking their average values. This approach naturally preserves conservation and remains stable in the presence of discontinuities, although it can introduce excessive numerical diffusion, smearing sharp features. More advanced methods include MUSCL schemes, which enhance FVM through slope reconstruction, and ENO/WENO schemes, which adaptively avoid discontinuities when constructing higher-order approximations. Whilst highly effective at capturing shocks, these methods are typically more complex to implement and computationally demanding.

A flexible intermediate approach is provided by the discontinuous Galerkin method. This method partitions the domain into elements and approximates the solution within each element by a polynomial. Unlike continuous Galerkin methods, continuity is not enforced across element interfaces; instead, discontinuities are permitted, with inter-element communication handled through numerical flux functions. This framework enables stable and physically consistent approximations while retaining high-order accuracy in smooth regions. A key advantage of the discontinuous Galerkin method is its adaptability. Increasing the polynomial degree or the order of the time-stepping scheme improves resolution of solution features. Furthermore, modifications such as limiters can be incorporated to suppress non-physical oscillations near discontinuities, thereby stabilising the approximation. The overall diffusivity of the method can be easily controlled through both the limiter and the numerical flux function, allowing a range of physical behaviours to be represented. Finally, since information is exchanged only between neighbouring elements, the method is well suited to parallel implementation and is widely regarded as one of the most parallel-friendly high-order schemes.

This project begins by outlining the construction of the discontinuous Galerkin method, including a derivation of the transport equation, a discussion of its weak formulation, and the use of Gaussian quadrature to approximate flux integrals. Prototypes of the numerical flux functions are introduced, and the details in implementations of time-stepping schemes are described. Next, we explore the nuance of the polynomial approximations inside the elements- not all polynomial approximations of equal degree are created the same. The major advantage of using polynomials with a Legendre or Lagrange structure doesn't arise from a significant improvement in approximation accuracy but rather a drastically streamlined implementation. The incorporation of higher-order time-stepping schemes, such as the fourth-order Runge-Kutta method, is also discussed in the context of the coupled spatial discretisation.

The method is first tested on a simple problem: linear advection with a periodic sinusoidal initial condition. The resulting errors are analysed, showing that for stable schemes the global error is governed by both the spatial polynomial degree and the order of the time stepper used. In the presence of discontinuities, however, the spatial error near the discontinuity does not improve with increasing polynomial degree. The use of limiters reduces local error near discontinuities at the cost of slightly increased error in smooth regions, typically resulting in an overall improvement. The importance of the Courant-Friedrichs-Lewy (CFL) condition, which requires the ratio of the spatial and temporal resolutions to be in accordance with the speed at which information travels between elements is also discussed. Failure to satisfy this condition in either continuous or discontinuous contexts leads to approximations with exponentially growing errors. Consequently, it is desirable to match the orders of spatial and temporal discretisation to avoid unnecessary computational cost.

Attention is then turned to discontinuous solutions. It is shown that even small discontinuities can lead to rapid degradation of the approximation when using an unmodified Galerkin scheme. This degradation arises from the formation of non-physical oscillations, known as Gibbs-type oscillations, near discontinuities. These oscillations are a consequence of the scheme's inherent structure and its attempt to anticipate steep gradients. Whilst higher ordered time steppers improve the representation of the shocks, they merely delay the onset of these oscillations and fail in preventing them from forming or propagating. To address this, limiters are introduced to suppress oscillations near discontinuities while preserving accuracy in smooth regions. Among the simplest are gener-

alised minmod limiters, which compare local slopes with those of neighbouring elements. These limiters balance stability and diffusion, with increased stability typically introducing greater numerical diffusion. This leads naturally to the question of how shocks should behave in different physical contexts. Some shocks, especially those within inviscid substrates, persist for a long period whereas others in more diffusive systems are smoothened much quicker. Generalised minmod limiters include a parameter that controls this behaviour, allowing the diffusivity of the approximation to be tuned. Similarly, numerical flux functions such as the generalised Lax-Friedrichs flux include parameters that introduce controlled artificial diffusion across element interfaces. Both of these smoothing parameters can be adjusted to strike different balances between stability and diffusive rates reflecting a wide variety of physical scenarios.

Finally, examples involving non-linear fluxes are considered, focusing in particular on the inviscid Burgers' equation. Such systems are characterised by the flattening of positive gradients and the steepening of negative gradients which causes local maxima to catch up with local minima. This may result in a shock formation with infinite gradients, even when the initial profile is smooth. For inviscid Burger's flow we show that the minimum time required for a shock to form from a smooth initial profile is determined exclusively by the initial profile itself with shallower initial negative gradients delaying the onset of a shock. The propagation of discontinuities is also examined, showing that their behaviour depends on orientation. In particular, jump up discontinuities are smoothed out, degrading the discontinuity's sharpness by eating into the upper portion of the discontinuity. On the other hand, jump down discontinuities remain sharp and distinctive. The Rankine-Hugoniot condition can be used to determine the speed at which a discontinuity will propagate; for Burger's flow, the speed is given by the average value of the quantity in the discontinuity with discontinuity's at higher values travelling faster.

Modelling transport systems containing shocks remains essential for understanding complex physical phenomena. When appropriately configured, the discontinuous Galerkin method provides a robust and flexible framework for approximating such systems. Although it is no longer the most advanced method available in all settings, it remains widely used and forms an important component of the numerical toolkit for convection-dominated transport problems.

2 Galerkin Method Formulation

2.1 Transport Equation

The transport equation provides a general mathematical description of the evolution of a conserved quantity within a given domain. It underpins the governing dynamics of numerous areas of applied mathematics, including fluid mechanics, molecular transport, and migration modelling in mathematical biology. Its derivation begins with a simple conservation principle:

Within a fixed region of space,

$$\text{change in quantity} = \text{flow of quantity into the region} + \text{source}. \quad (1)$$

In fluid mechanics, the quantity under consideration is typically mass; in molecular transport it may represent the number of ions; and in migration modelling it may denote the population of a given species. The source term represents the net production or destruction of the quantity. For example, in molecular transport it may correspond to chemical reactions, while in migration modelling it may represent births or deaths.

Equation (1) may equivalently be written as

$$\text{change in quantity} = -\text{flow of quantity out of the region} + \text{source}. \quad (2)$$

If u denotes the density of the quantity and V the region of space under consideration, then equation (2) becomes

$$\frac{d}{dt} \int_V u \, dV = - \int_{\partial V} \mathbf{j} \cdot \mathbf{n} \, dS + \int_V \Sigma \, dV, \quad (3)$$

where Σ represents the source term, dS denotes the surface measure on ∂V , $\mathbf{n}$ is the outward-pointing unit normal vector to ∂V , and $\mathbf{j}$ is the flux vector governing the transport of u .

Before proceeding, we introduce the divergence theorem, which will be used repeatedly in what follows.

Divergence Theorem: Let $V \subset \mathbb{R}^n$ and let $\mathbf{F}$ be a sufficiently smooth vector field. Then

$$\int_V \nabla \cdot \mathbf{F} dV = \int_{\partial V} \mathbf{F} \cdot \mathbf{n} dS.$$

Applying the divergence theorem to the flux term, using Leibniz's rule to bring the time derivative inside the integral, and rearranging terms yields

$$\int_V \left(\frac{\partial u}{\partial t} + \nabla \cdot \mathbf{j} - \Sigma \right) dV = 0. \quad (4)$$

Since the choice of the region V is arbitrary, the integrand must vanish pointwise. Hence, the transport equation is given by

$$\frac{\partial u}{\partial t} + \nabla \cdot \mathbf{j} - \Sigma = 0. \quad (5)$$

The transport equation therefore arises directly from a fundamental conservation principle and provides a unified framework for modelling the evolution of physical quantities. Depending on the choice of the conserved quantity, it may lead to a variety of well-known partial differential equations; for instance, conservation of energy yields the heat equation, while conservation of momentum leads to the Navier–Stokes equations.

The flux vector $\mathbf{j}$ may, in general, be decomposed into a convective component $\mathbf{j}_{\text{conv}}$ and a diffusive component $\mathbf{j}_{\text{diff}}$, given by

$$\mathbf{j}_{\text{conv}} = u\mathbf{v}, \quad \mathbf{j}_{\text{diff}} = -D\nabla u,$$

where $\mathbf{v}$ denotes the velocity field of the surrounding medium and D is a diffusion coefficient. The convective flux represents transport due to bulk motion of the medium, while the diffusive flux models transport driven by gradients in the quantity itself, acting in the direction of decreasing density. The total flux is therefore

$$\mathbf{j} = \mathbf{j}_{\text{conv}} + \mathbf{j}_{\text{diff}}. \quad (6)$$

In convection-dominated problems, the diffusive contribution is negligible in comparison with the convective contribution, so that $\mathbf{j}_{\text{diff}} \ll \mathbf{j}_{\text{conv}}$ and consequently $\mathbf{j} \approx \mathbf{j}_{\text{conv}}$.

Writing the convective flux as a nonlinear function of u , $\mathbf{j}_{\text{conv}} = u\mathbf{v} = \mathbf{f}(u)$, the transport equation for convection-dominated problems may be expressed as

$$\frac{\partial u}{\partial t} + \nabla \cdot \mathbf{f}(u) - \Sigma = 0. \quad (7)$$

2.2 Weak Form

We restrict attention to problems in which the transported quantity varies in one spatial dimension only, denoted by x , and where the effects of sources and sinks are negligible, so that $\Sigma \approx 0$. Under these assumptions, equation (7) reduces to the one-dimensional conservation law

$$\frac{\partial u}{\partial t} + \frac{\partial f(u)}{\partial x} = 0. \quad (8)$$

We consider initial value problems for which the initial condition $u(x, t = 0)$ is prescribed across the entire spatial domain. For such problems, and under appropriate regularity assumptions on f , this specification is sufficient to guarantee uniqueness of the solution, at least up to the formation of shocks. Physically, this corresponds to a situation in which the initial configuration of a system is known and the subsequent temporal evolution is sought.

A function u that satisfies equation (8) pointwise, together with the prescribed initial condition, is said to be a *strong solution*. In many applications, however, solutions may fail to possess sufficient regularity for the derivatives in equation (8) to exist in the classical sense. This motivates the introduction of a weaker formulation of the problem.

To derive the weak form, equation (8) is multiplied by a sufficiently smooth test function v with compact support and integrated over the spatial domain, yielding

$$\int \left(\frac{\partial u}{\partial t} + \frac{\partial f(u)}{\partial x} \right) v \, dx = 0. \quad (9)$$

Any strong solution of equation (8) immediately satisfies equation (9) for all admissible test functions v . Conversely, if u satisfies equation (9) for all such v and the initial condition is satisfied in an appropriate sense, then u coincides with a strong solution wherever sufficient regularity is present. More generally, the weak formulation allows solutions to be defined even when classical derivatives do not exist, making it particularly suitable for problems involving sharp gradients or discontinuities.

2.3 Galerkin Approximation

Solving the weak formulation over an infinite-dimensional space of trial and test functions is intractable in practice. The Galerkin method reduces the problem to a finite-dimensional setting by restricting both the approximate solution and the test functions to a prescribed discrete space.

We begin by partitioning the one-dimensional domain into a finite collection of elements. For each element e we denote the left and right boundaries by x_0^e and x_1^e respectively. The physical domain is continuous, so $x_1^e = x_0^{e+1}$, but the approximation is allowed to be discontinuous and hence, in general, $u_1^e \neq u_0^{e+1}$, where $u_i^e = u(x_i^e)$.

For a linear discontinuous Galerkin approximation, the discrete space consists of functions that are linear on each element but may jump across interfaces; see Figure 1 (a). A convenient basis is given by two local functions per element, defined by

$$\psi_i^e = \begin{cases} 1 & \text{at } x_i^e \text{ and linear within element } e, \\ 0 & \text{at } x_j^e, j \neq i, \text{ and outside } e. \end{cases} \quad (10)$$

In coordinates,

$$\begin{aligned} \psi_0^e(x) &= \begin{cases} \frac{x_1^e - x}{x_1^e - x_0^e} & x \in e, \\ 0 & \text{otherwise,} \end{cases} \\ \psi_1^e(x) &= \begin{cases} \frac{x - x_0^e}{x_1^e - x_0^e} & x \in e, \\ 0 & \text{otherwise.} \end{cases} \end{aligned} \quad (11)$$

Any discrete test function can therefore be written as

$$v = \sum_{e=0}^{n-1} (\alpha_0^e \psi_0^e + \alpha_1^e \psi_1^e). \quad (12)$$

Since the weak formulation is linear in v , it is sufficient to test the equation using each basis function individually.

We now impose the same representation for the approximate solution,

$$u = \sum_{e=0}^{n-1} (u_0^e \psi_0^e + u_1^e \psi_1^e), \quad (13)$$

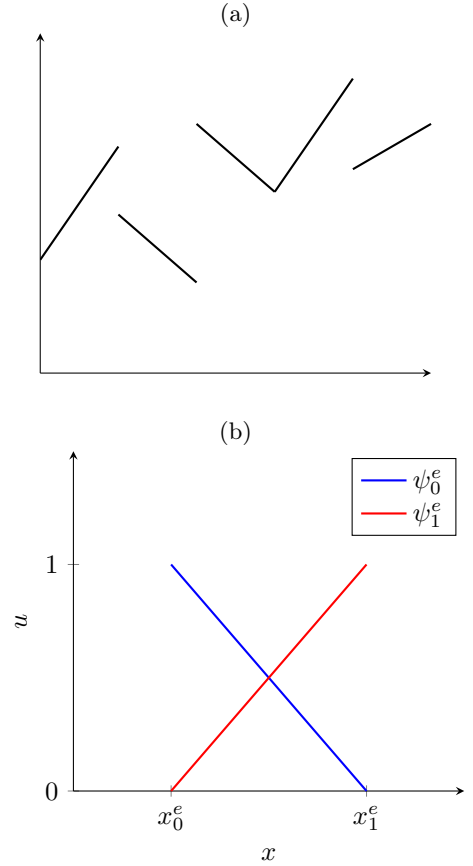


Figure 1: (a) Discontinuous linear spline. (b) Local linear basis functions on element e .

where the coefficients u_i^e are time dependent. They are determined initially by projection or interpolation of the given initial condition, after which the scheme provides their temporal evolution.

Because each basis function associated with element e vanishes outside that element, the weak form reduces to

$$\int_{x_0^e}^{x_1^e} \left(\frac{\partial u}{\partial t} + \frac{\partial f(u)}{\partial x} \right) v \, dx = 0. \quad (14)$$

If discontinuities are present, $\partial_x f(u)$ need not exist pointwise and must be interpreted in a weak sense. It is therefore advantageous to transfer the derivative onto the test function. Even when the solution is smooth, this reformulation is beneficial: $f(u)$ encodes more information than $\frac{\partial f}{\partial x}$, so numerical approximations of integrals involving the first term are going to be significantly more stable. Moreover, the procedure naturally exposes boundary contributions, which will later provide a systematic mechanism for coupling neighbouring elements. Performing integration by parts yields

$$\int_{x_0^e}^{x_1^e} \left(\frac{\partial u}{\partial t} v - f(u) \frac{\partial v}{\partial x} \right) dx + [f(u)v]_{x_0^e}^{x_1^e} = 0.$$

To allow neighbouring elements to interact, the physical flux at the interfaces is replaced by a numerical flux. Without this modification, each element would evolve independently, since the weak formulation involves only information internal to the element. The numerical flux introduces a controlled exchange of information across interfaces by making the boundary contribution depend on states from adjacent elements. In this way the global scheme remains locally defined while still representing propagation through the entire mesh. The formulation becomes

$$\int_{x_0^e}^{x_1^e} \left(\frac{\partial u}{\partial t} v - f(u) \frac{\partial v}{\partial x} \right) dx + h(u_1^e, u_0^{e+1})v(x_1^e) - h(u_1^{e-1}, u_0^e)v(x_0^e) = 0. \quad (15)$$

We now choose $v = \psi_0^e$ and $v = \psi_1^e$ in turn. Using the boundary values of the basis functions leads to

$$\int_{x_0^e}^{x_1^e} \frac{\partial u}{\partial t} \psi_0^e \, dx = \int_{x_0^e}^{x_1^e} f(u) \frac{\partial \psi_0^e}{\partial x} \, dx + h(u_1^{e-1}, u_0^e), \quad (16)$$

$$\int_{x_0^e}^{x_1^e} \frac{\partial u}{\partial t} \psi_1^e \, dx = \int_{x_0^e}^{x_1^e} f(u) \frac{\partial \psi_1^e}{\partial x} \, dx - h(u_1^e, u_0^{e+1}). \quad (17)$$

Since the basis functions are time independent,

$$\frac{\partial u}{\partial t} = \dot{u}_0^e \psi_0^e + \dot{u}_1^e \psi_1^e.$$

Substitution produces,

$$\dot{u}_0^e \int_{x_0^e}^{x_1^e} \psi_0^e \psi_0^e \, dx + \dot{u}_1^e \int_{x_0^e}^{x_1^e} \psi_0^e \psi_1^e \, dx = \int_{x_0^e}^{x_1^e} f(u) \frac{\partial \psi_0^e}{\partial x} \, dx + h(u_1^{e-1}, u_0^e), \quad (18)$$

$$\dot{u}_0^e \int_{x_0^e}^{x_1^e} \psi_0^e \psi_1^e \, dx + \dot{u}_1^e \int_{x_0^e}^{x_1^e} \psi_1^e \psi_1^e \, dx = \int_{x_0^e}^{x_1^e} f(u) \frac{\partial \psi_1^e}{\partial x} \, dx - h(u_1^e, u_0^{e+1}). \quad (19)$$

giving the linear system

$$M^e \dot{\mathbf{U}}^e = \mathbf{F}^e(\mathbf{U}^e) + \mathbf{H}^e(\mathbf{U}^{e-1}, \mathbf{U}^e, \mathbf{U}^{e+1}), \quad (20)$$

where

$$M_{ij}^e = \int_{x_0^e}^{x_1^e} \psi_i^e \psi_j^e \, dx, \quad (21)$$

$$\mathbf{F}_i^e(\mathbf{U}^e) = \int_{x_0^e}^{x_1^e} f(u) \frac{\partial \psi_i^e}{\partial x} \, dx, \quad (22)$$

and

$$\mathbf{H}^e = \begin{pmatrix} h(u_1^{e-1}, u_0^e) \\ -h(u_1^e, u_0^{e+1}) \end{pmatrix}. \quad (23)$$

Multiplying by $(M^e)^{-1}$ yields the evolution equations

$$\dot{\mathbf{U}}^e = (M^e)^{-1} [\mathbf{F}^e(\mathbf{U}^e) + \mathbf{H}^e(\mathbf{U}^{e-1}, \mathbf{U}^e, \mathbf{U}^{e+1})]. \quad (24)$$

The mass matrix may be evaluated analytically, whereas the volume integrals in $\mathbf{F}^e$ are typically computed by numerical quadrature.

2.3.1 Mass Matrix Computation

It is possible to compute the entries of the mass matrix directly. However, since the evaluation of the flux vector will later employ reparametrised basis functions, it is convenient to introduce this mapping here. The domain over a single element, say e , can be written in the linear form

$$x = \frac{1}{2} [(x_0^e + x_1^e) + s(x_1^e - x_0^e)], \quad s \in [-1, 1]. \quad (25)$$

Similarly, the solution may be expressed in terms of s as

$$u = \frac{1}{2} [(u_0 + u_1) + s(u_1 - u_0)], \quad s \in [-1, 1], \quad (26)$$

and the basis functions become

$$\psi_0^e = \frac{1}{2}(1 - s), \quad \psi_1^e = \frac{1}{2}(1 + s), \quad s \in [-1, 1]. \quad (27)$$

Under this transformation,

$$dx = \frac{x_1^e - x_0^e}{2} ds,$$

and hence the mass matrix entries may be written as

$$M_{ij}^e = \frac{x_1^e - x_0^e}{2} \int_{-1}^1 \psi_i^e \psi_j^e ds. \quad (28)$$

We therefore obtain

$$M_{00}^e = \frac{x_1^e - x_0^e}{8} \int_{-1}^1 (1 - s)^2 ds = \frac{x_1^e - x_0^e}{3}, \quad (29)$$

$$M_{11}^e = \frac{x_1^e - x_0^e}{8} \int_{-1}^1 (1 + s)^2 ds = \frac{x_1^e - x_0^e}{3}, \quad (30)$$

and

$$M_{01}^e = M_{10}^e = \frac{x_1^e - x_0^e}{8} \int_{-1}^1 (1 - s^2) ds = \frac{x_1^e - x_0^e}{6}. \quad (31)$$

Combining these results gives

$$M^e = \frac{x_1^e - x_0^e}{6} \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}. \quad (32)$$

If the spatial step size is uniform across all elements, denoted by h , then

$$M = \frac{h}{6} \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} \quad \text{for every element.} \quad (33)$$

Inverting M yields

$$M^{-1} = \frac{2}{h} \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}. \quad (34)$$

2.3.2 Flux Vector Approximation

We begin the computation of the flux vector from

$$\mathbf{F}^e(\mathbf{U}^e) = \frac{1}{x_1^e - x_0^e} \int_{x_0^e}^{x_1^e} f(u) dx \begin{pmatrix} -1 \\ 1 \end{pmatrix}. \quad (35)$$

The principal term requiring evaluation is the integral $\int_{x_0^e}^{x_1^e} f(u) dx$. Before discussing how this may be approximated, we again map each element to the reference interval using the parametrisation in s . This yields

$$\mathbf{F}^e(\mathbf{U}^e) = \frac{1}{2} \int_{-1}^1 f(u) ds \begin{pmatrix} -1 \\ 1 \end{pmatrix}, \quad (36)$$

and therefore the task reduces to approximating $\int_{-1}^1 f(u) ds$.

Recall that the values at the endpoints of the interval are known, namely $u(s = -1) = u_0^e$ and $u(s = 1) = u_1^e$, and that within the element u is taken to be the linear interpolant of these boundary values. In many simple cases the integral may be computed analytically. However, it is advantageous to develop a numerical framework that applies uniformly to a wide class of flux functions and does not rely on explicit integration.

At first sight, the trapezoidal rule appears attractive due to its simplicity. Nevertheless, the integral must be evaluated separately on every element, and obtaining high accuracy from the trapezoidal rule would require a fine internal resolution, leading to a significant computational cost. Moreover, if $f''(u)$ is large, low-resolution approximations may be poor.

A more efficient alternative is provided by Gaussian quadrature. This approach achieves high accuracy with relatively few evaluation points, provided the flux is well approximated by a polynomial. In particular, an n -point Gaussian quadrature rule integrates polynomials of degree up to $2n - 1$ exactly. In this project we consider two flux functions: linear advection with $f(u) = u$, and inviscid Burgers' equation with $f(u) = \frac{1}{2}u^2$. A two-point Gaussian rule is therefore sufficient for both cases.

The two-point Gaussian quadrature approximation is

$$\int_{-1}^1 f(u) ds \approx f\left(-\frac{1}{\sqrt{3}}\right) + f\left(\frac{1}{\sqrt{3}}\right), \quad (37)$$

and hence the flux vector is approximated by

$$\mathbf{F}^e(\mathbf{U}^e) = \frac{1}{2} \left[f\left(-\frac{1}{\sqrt{3}}\right) + f\left(\frac{1}{\sqrt{3}}\right) \right] \begin{pmatrix} -1 \\ 1 \end{pmatrix}, \quad (38)$$

where the flux values are obtained by evaluating $u(s)$ via the linear interpolation between u_0^e and u_1^e .

2.3.3 Lax-Friedrichs Numerical Flux

The numerical flux function, $h(a, b)$, represents the flux at the boundary of an element while incorporating information from its neighbouring element. Its role is to couple otherwise independent elements through the interface contribution. Numerous numerical fluxes exist, each reflecting different physical and numerical considerations, particularly regarding stability and the degree of interaction between neighbouring elements.

A widely used and robust choice within the discontinuous Galerkin framework is the Lax-Friedrichs numerical flux, defined by

$$h(a, b) = \frac{1}{2} (f(a) + f(b)) - \frac{1}{2} \max_{a \leq \zeta \leq b} |f'(\zeta)| (b - a). \quad (39)$$

The Lax-Friedrichs flux can be interpreted as the result of adding artificial viscosity to the original conservation law. Specifically, it corresponds to a discretisation of a convection-diffusion system in which the diffusion coefficient is proportional to $\max_{a \leq \zeta \leq b} |f'(\zeta)|$. In this way, the

method stabilises the hyperbolic problem by introducing controlled dissipation at element interfaces.

The first term, $\frac{1}{2}(f(a) + f(b))$, is known as the central flux. It represents the arithmetic mean of the flux evaluated on either side of the interface and treats both neighbouring states symmetrically. On its own, this term produces an energy-neutral (non-dissipative) scheme. Although consistent, such a scheme does not damp high-frequency modes and is therefore prone to unstable oscillations in the presence of discontinuities.

The second term,

$$-\frac{1}{2} \max_{a \leq \zeta \leq b} |f'(\zeta)|(b-a),$$

acts as a jump penalty and introduces artificial viscosity proportional to the magnitude of the discontinuity across the interface. This term biases information transfer in the direction of wave propagation and therefore introduces an upwind character to the scheme. Its presence ensures that energy is dissipated at interfaces, suppressing oscillations that would otherwise develop near discontinuities.

2.3.4 Algorithm Implementation

Recall that the one-dimensional domain is partitioned into n elements, each possessing two degrees of freedom corresponding to its left and right nodal values. The initial condition provides the starting values of these nodal quantities. To advance the solution in time, a time-stepping scheme must be applied. The simplest choice is the first-order forward Euler method, which updates each element according to

$$\mathbf{U}^{e,t+\Delta t} = \mathbf{U}^{e,t} + \Delta t \dot{\mathbf{U}}^{e,t}. \quad (40)$$

A crucial implementation detail is that the evaluation of $\dot{\mathbf{U}}^{e,t}$ depends not only on $\mathbf{U}^{e,t}$ but also on the neighbouring states $\mathbf{U}^{e-1,t}$ and $\mathbf{U}^{e+1,t}$ through the numerical flux. Consequently, all spatial derivatives must be computed using the solution at time level t before any updates to $t + \Delta t$ are committed. Updating element values in place would overwrite data required for neighbouring flux evaluations, leading to an inconsistent time advancement.

To avoid this issue, separate storage must be maintained for the updated solution $\mathbf{U}^{e,t+\Delta t}$ (or equivalently for $\dot{\mathbf{U}}^{e,t}$). Only once all elements have been advanced consistently may the solution vectors be overwritten.

Finally, periodic boundary conditions are imposed by defining the left neighbour of the first element with the last element and the right neighbour of the last element with the first, allowing information to exit one boundary and re-enter at the other. Such a configuration is particularly useful for assessing long-time stability and accuracy without introducing additional boundary treatment effects.

3 Example: Continuous Advection

Our first example considers the simplest possible setting and provides a natural entry point for examining the behaviour of the Galerkin algorithm. We study the Galerkin approximation of the initial value problem

$$\left. \begin{aligned} \frac{\partial u}{\partial t} + \frac{\partial u}{\partial x} &= 0, \\ u(x, 0) &= 1.5 + \sin(x), \end{aligned} \right\} \quad 0 \leq x \leq 2\pi, \quad t \geq 0. \quad (41)$$

This is the one-dimensional linear advection equation with wave speed $c = 1$ and a smooth sinusoidal initial condition. In the numerical implementation, periodic boundary conditions are imposed by identifying the first and last elements, ensuring that waves exiting one boundary re-enter at the other.

A system of this form arises naturally when studying the transport of a passive scalar by a uniform flow, such as the advection of a small temperature or concentration perturbation in a fluid moving with constant velocity. Sinusoidal initial conditions are also commonly used in numerical analysis because they represent a single Fourier mode, allowing the propagation and phase accuracy of the numerical method to be examined clearly. More generally, since arbitrary smooth signals

can be decomposed into Fourier modes, understanding how a numerical scheme advects a sinusoid provides insight into how it will behave for more complex smooth initial data.

Before analysing the numerical solution, it is useful to clarify the plotting convention adopted for Galerkin approximations. Strictly speaking, the discrete solution is a discontinuous linear spline. However, rather than plotting the full piecewise representation, a single representative point is shown for each element, namely the mid-point value $(x(0), u(0))$ in reference coordinates. This reduces visual clutter and suppresses the apparent discontinuities at element interfaces, making the overall behaviour of the solution easier to interpret. Figure 2(a) illustrates the construction of the sinusoidal initial condition using seven elements and the corresponding midpoint representation.

The linear advection equation admits a closed-form analytical solution. For wave speed c , the general solution is

$$u(x, t) = u_0(x - ct), \quad (42)$$

where u_0 denotes the initial condition. This general solution to the equation can be validated easily by computing both of the spatial and temporal derivatives in turn. Applying this to the present problem yields

$$u(x, t) = 1.5 + \sin(x - t). \quad (43)$$

Thus, the exact solution is a pure translation of the initial sinusoidal profile to the right with unit speed and without change of amplitude or shape.

Figure 2(b) demonstrates that the Galerkin approximation accurately reproduces this behaviour for the smooth, continuous example considered here.

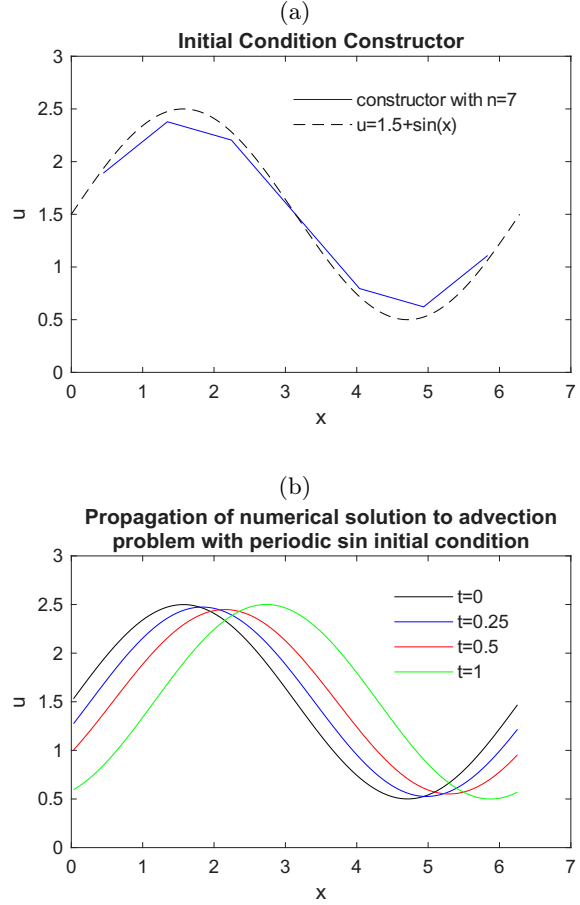


Figure 2: (a) Illustration of the construction of the sinusoidal initial condition. (b) Propagation of the numerical solution of an advection system with a sinusoidal initial condition. 100 elements were used with a timestep of 0.001 sec.

4 Improving the Order of the Galerkin Method and the Time Stepping Algorithm

The implementation outlined above is intentionally elementary. It contains no modifications such as limiters and employs both a linear Galerkin approximation, arising from the use of linear basis functions, and a first order time stepping scheme. For simple continuous solutions this basic construction is often sufficient (see the example below). However, in more demanding scenarios higher order Galerkin approximations and time integrators are typically required. In this section we discuss the principal considerations involved in increasing the order of the spatial discretisation and the time stepping algorithm.

Recall that for the linear Galerkin method, the solution within each element is approximated by a linear function determined by two reference degrees of freedom. Assembling these local approximations across the domain produces a discontinuous linear spline at each fixed time. Increasing the order of the Galerkin method consists of replacing the linear approximation on each element by a polynomial of higher degree together with additional degrees of freedom. Irrespective of the order employed, equation (15) remains the starting point. As before, the test function v is taken in

turn to be each basis function, yielding a system of ordinary differential equations for $\dot{\mathbf{U}}$. Although the choice of basis is not unique, an n^{th} order Galerkin method requires a set of basis functions spanning $\{1, x, \dots, x^n\}$ and therefore at least $n + 1$ basis functions per element. Consequently, the system corresponding to equation (24) is $(n + 1)$ -dimensional for an n^{th} order method.

Whilst in theory any basis would work, in practice, how the choice of basis influences the algebraic structure of each term in equation (24) should be considered. In particular, it affects conditioning and hence the sensitivity of the scheme to roundoff error. A judicious choice of basis can therefore improve numerical robustness.

The entries of the numerical flux vector are

$$\mathbf{H}_i^e = h(u_1^e, u_0^{e+1})\psi_i^e(x_1^e) - h(u_1^{e-1}, u_0^e)\psi_i^e(x_0^e), \quad (44)$$

so the basis functions enter only through their values at the element endpoints. Choosing basis functions that take values 0 or 1 at these endpoints simplifies the scaling of the boundary contributions and improves numerical stability in practice. Both Legendre polynomial bases (in modal form) and Lagrange polynomial bases constructed with Gauss-Lobatto quadrature possess this property.

The flux vector is given by

$$\mathbf{F}_i^e = \int_{x_0^e}^{x_1^e} f(u) \frac{\partial \psi_i^e}{\partial x} dx. \quad (45)$$

If the degree of the polynomial basis function increases, then so does the degree of the integrand. When Gaussian quadrature is employed to approximate this integral, a sufficient number of quadrature points must be used to maintain the desired order of accuracy. In a straightforward implementation this requires evaluating each basis function at each quadrature point, leading to an $\mathcal{O}(n^2)$ cost per element for an n^{th} order method.

For Legendre polynomial bases this cost remains $\mathcal{O}(n^2)$, as the solution must be reconstructed at quadrature points from its modal coefficients. In contrast, if Lagrange basis functions are constructed with the same quadrature nodes as those used in the evaluation of the integrals, then the solution values at quadrature points coincide with the stored degrees of freedom. The evaluation of the nonlinear flux $f(u)$ therefore becomes a pointwise operation, and the reconstruction step is avoided yielding significant practical simplification. In order to preserve the design order of accuracy, the quadrature rule must be chosen so that it integrates the relevant polynomial degrees with sufficient precision which requires consideration of the degree of the flux function.

The mass matrix entries are

$$M_{ij}^e = \int_{x_0^e}^{x_1^e} \psi_i^e \psi_j^e dx. \quad (46)$$

For a general basis satisfying only the spanning condition, the mass matrix is dense. Since the computation of $\dot{\mathbf{U}}$ requires solving a linear system involving M^e , poorly structured mass matrices can become a computational bottleneck, especially if the degree is large, even if their inverses are formed only once.

Legendre polynomials satisfy

$$\int_{-1}^1 P_i P_j dx = 0 \quad \text{for } i \neq j,$$

and hence produce diagonal mass matrices under exact integration. This property accounts for their classical use in Galerkin methods. Alternatively, Lagrange polynomials constructed at Gauss-Lobatto nodes satisfy $\psi_i(x_k)\psi_j(x_k) = 0$ for $i \neq j$ at each quadrature node x_k . When the same Gauss-Lobatto quadrature rule is used to approximate the integrals in the mass matrices, the resulting mass matrix is diagonal under quadrature and provides a high quality approximation to the exact mass matrix. For this reason, Lagrange polynomials with Gauss-Lobatto quadrature provide good basis functions for modern Galerkin methods. For smooth solutions, increasing the polynomial degree improves the spatial accuracy; however, for discontinuous solutions the global convergence order is limited, as discussed in the corresponding section.

With respect to time integration, the forward Euler method is the simplest explicit scheme and is often adequate for smooth solutions over moderate time intervals. Nevertheless, higher order

time integrators are generally required when increased temporal accuracy is desired or when the spatial approximation order is raised. A classical example is the fourth order Runge-Kutta scheme:

$$\begin{aligned}
\mathbf{k}_1^{e,t} &= \dot{\mathbf{U}}(\mathbf{U}^{e,t}, \mathbf{U}^{e-1,t}, \mathbf{U}^{e+1,t}), \\
\mathbf{k}_2^{e,t} &= \dot{\mathbf{U}}(\mathbf{U}^{e,t} + \frac{\Delta t}{2} \mathbf{k}_1^{e,t}, \mathbf{U}^{e-1,t} + \frac{\Delta t}{2} \mathbf{k}_1^{e-1,t}, \mathbf{U}^{e+1,t} + \frac{\Delta t}{2} \mathbf{k}_1^{e+1,t}), \\
\mathbf{k}_3^{e,t} &= \dot{\mathbf{U}}(\mathbf{U}^{e,t} + \frac{\Delta t}{2} \mathbf{k}_2^{e,t}, \mathbf{U}^{e-1,t} + \frac{\Delta t}{2} \mathbf{k}_2^{e-1,t}, \mathbf{U}^{e+1,t} + \frac{\Delta t}{2} \mathbf{k}_2^{e+1,t}), \\
\mathbf{k}_4^{e,t} &= \dot{\mathbf{U}}(\mathbf{U}^{e,t} + \Delta t \mathbf{k}_3^{e,t}, \mathbf{U}^{e-1,t} + \Delta t \mathbf{k}_3^{e-1,t}, \mathbf{U}^{e+1,t} + \Delta t \mathbf{k}_3^{e+1,t}), \\
\mathbf{U}^{e,t+\Delta t} &= \mathbf{U}^{e,t} + \frac{\Delta t}{6} (\mathbf{k}_1^{e,t} + 2\mathbf{k}_2^{e,t} + 2\mathbf{k}_3^{e,t} + \mathbf{k}_4^{e,t}).
\end{aligned}$$

The stage vectors $\mathbf{k}_i^{e,t}$ must be computed sequentially, as each stage depends on the preceding one. Since $\dot{\mathbf{U}}$ involves neighbouring elements, all stage values must be available consistently across the mesh before advancing to the next stage. In practice, separate storage is allocated for each stage vector, and the spatial operator is evaluated four times per time step, each time with the appropriately updated intermediate state. This procedure increases computational cost but yields fourth order accuracy in time, provided the Courant-Friedrichs-Lewy condition is respected.

It can also be helpful to use implicit time steppers such as the first order Euler Backward. For a straightforward advection system, $\mathbf{U}^{e,t+\Delta t}$ can be computed directly from $\mathbf{U}^{e,t}$ as in the explicit case. For non linear system, $\mathbf{U}^{e,t+\Delta t}$ is guessed and then iterated towards the true solution via a Newton method. Since this has to be done at each new time step, implicit methods aren't as practical in non linear cases. The advantage of using implicit methods is that their stability conditions aren't as tight as their explicit counterparts. That is, their approximations remain stable for a much larger region of spatial and temporal resolutions, despite being first order.

5 Error Analysis

The error in a discontinuous Galerkin approximation arises from three distinct but interacting sources: the spatial discretisation, the temporal discretisation, and the stability properties of the fully discrete scheme. The relative importance of each contribution depends on the regularity of the exact solution, the polynomial degree of the approximation, the order of the time stepping method, and whether nonlinear stabilisation such as limiting is employed.

Let u denote the exact solution and u_h its discontinuous Galerkin approximation at a fixed time T . The error is measured in the L_2 norm,

$$\|u - u_h\|_{L_2(a,b)}^2 = \int_a^b (u - u_h)^2 dx. \quad (47)$$

Throughout, we assume that the numerical flux is consistent and that appropriate boundary conditions are imposed.

5.1 Spatial Discretisation Error

Consider a p -th order discontinuous Galerkin method, meaning that within each element the solution is approximated by polynomials of degree at most p , giving $p+1$ local degrees of freedom. Even before time discretisation is introduced, replacing u by a piecewise polynomial incurs a projection error.

If the exact solution u is sufficiently smooth, standard approximation theory yields

$$\|u - u_h\|_{L_2} = \mathcal{O}(h^{p+1}), \quad (48)$$

provided the numerical flux is consistent and the scheme is stable. In particular, the semi-discrete residual and hence the computed time derivative $\dot{\mathbf{U}}$ inherit the same order of accuracy. Thus, for smooth solutions, increasing the polynomial degree improves the spatial convergence rate.

The situation differs when u contains discontinuities. In this case, no polynomial approximation can achieve higher than first-order accuracy in a neighbourhood of the discontinuity. Globally, one typically observes

$$\|u - u_h\|_{L_2} = \mathcal{O}(h^{1/2}), \quad (49)$$

for linear hyperbolic problems without additional stabilisation. This reduced rate reflects the fact that discontinuities and the oscillations that arise from them dominate the accuracy (or inaccuracy) of the approximation.

If a limiter is applied, oscillations near discontinuities are suppressed at the expense of locally reducing the polynomial degree. In that case, one generally observes

$$\|u - u_h\|_{L_2} = \mathcal{O}(h), \quad (50)$$

globally, while retaining $\mathcal{O}(h^{p+1})$ accuracy in smooth regions. Consequently, for problems with shocks, increasing p alone does not improve the global convergence rate unless limiting is carefully designed to preserve high-order accuracy away from discontinuities.

5.2 Temporal Discretisation Error

After spatial discretisation, the discontinuous Galerkin method yields a system of ordinary differential equations,

$$M \frac{d\mathbf{U}}{dt} = R(\mathbf{U}), \quad (51)$$

where M is the mass matrix and R the spatial residual. This system is advanced in time using a k -th order time stepping scheme.

For sufficiently smooth solutions in time and a stable scheme, a k -th order method produces a global temporal error of order

$$\mathcal{O}(\Delta t^k). \quad (52)$$

For example, forward Euler is first order, whereas a classical fourth-order Runge–Kutta method yields $\mathcal{O}(\Delta t^4)$ temporal accuracy.

Unlike the spatial discretisation, improving the order of the time stepping method increases the temporal convergence rate irrespective of whether the solution is continuous or discontinuous. However, if the spatial error dominates, upgrading the time integrator will not reduce the total error.

5.3 Combined Error Estimate

Under suitable stability assumptions and for sufficiently smooth solutions, the fully discrete error satisfies

$$\|u - u_h\|_{L_2} = \mathcal{O}(h^{p+1}) + \mathcal{O}(\Delta t^k). \quad (53)$$

Thus the overall accuracy is governed by the larger of the two contributions. If Δt is chosen independently of h , one of the terms will dominate.

5.4 Stability and the CFL Condition

For explicit time stepping applied to hyperbolic conservation laws, stability requires the Courant–Friedrichs–Lewy condition

$$C = \frac{\max_{a \leq \zeta \leq b} |f'(\zeta)| \Delta t}{h} \leq C_{\max}, \quad (54)$$

where C is the Courant number and $C_{\max}$ depends on the polynomial degree p , the numerical flux, the time stepping method and the presence of limiters. The CFL condition governs stability for both continuous and discontinuous solutions.

Heuristically, the condition ensures that information does not travel across more than one element during a single time step. If it is violated, numerical errors are amplified and the discrete solution becomes unstable.

For discontinuous Galerkin methods with explicit Runge–Kutta schemes, $C_{\max}$ decreases as p increases, reflecting the presence of higher-frequency modes in higher-order polynomial spaces. Conversely, higher-order time stepping methods typically admit larger admissible values of $C_{\max}$ than lower-order explicit methods. Implicit methods remove the strict CFL restriction for stability, though accuracy considerations still limit the practical size of Δt . If limiters are used, then they can also decrease the value of $C_{\max}$.

When the CFL condition is enforced, one typically has $\Delta t = \mathcal{O}(h)$. Substituting this into the combined error estimate yields

$$\|u - u_h\|_{L_2} = \mathcal{O}(h^{\min(p+1, k)}). \quad (55)$$

Hence the overall convergence rate is limited by the smaller of the spatial and temporal orders.

If the CFL condition is violated for an explicit scheme, the error may grow exponentially in time. In that regime, one may observe behaviour of the form

$$\|u(T) - u_h(T)\|_{L_2} \approx e^{\lambda T} (h^{p+1} + \Delta t^k), \quad (56)$$

with $\lambda > 0$ determined by the unstable amplification factor.

5.5 Resolution Regimes

To illustrate the interaction between spatial and temporal resolution, consider the simplest case discussed earlier: system (41) evaluated at the final time $T = 1$, approximated using a linear Galerkin method ($p = 1$) together with the first-order forward Euler time stepping scheme. In this setting the spatial discretisation error scales as $\mathcal{O}(h^2)$ while the temporal discretisation error scales as $\mathcal{O}(\Delta t)$, provided the stability condition is satisfied. Consequently,

$$\text{Error} = \|u(T = 1) - u_h(T = 1)\|_2 \sim \mathcal{O}(h^2) + \mathcal{O}(\Delta t). \quad (57)$$

Assuming $0 < h < 1$, it follows that $h^2 < h$. This observation allows the different error regimes to be interpreted in terms of the relative sizes of h and Δt . In particular, three qualitatively different cases may arise:

- $h \ll \Delta t$. In this regime the CFL condition is violated, since Δt is too large relative to the spatial resolution. As a consequence the numerical approximation becomes unstable and the error increases rapidly. This behaviour is illustrated by the rapidly growing error in Figure 3(a) on the left-hand side and in Figure 3(b) on the right-hand side.
- $h^2 \ll \Delta t \ll h$.

Here the CFL condition is satisfied, but the temporal resolution is still relatively coarse compared with the spatial resolution. The error is therefore dominated by the temporal discretisation term. In this case $\text{Error} \sim \Delta t$, and hence $\log(\text{Error}) \sim \log(\Delta t)$. Consequently, a log–log plot of the error against Δt exhibits a region of gradient one. This behaviour corresponds to the flat section visible in Figure 3(a) and the section of gradient one in Figure 3(b).

- $\Delta t \ll h^2$. The CFL condition is again satisfied, but the temporal resolution is now sufficiently fine that the spatial discretisation error dominates. In this situation $\text{Error} \sim h^2$, so that $\log(\text{Error}) \sim 2 \log(h)$. Accordingly, a log–log plot of the error against h displays a region of gradient two. This behaviour corresponds to the flat section in Figure 3(b) and the section of gradient two in Figure 3(a).

These regimes can also be interpreted simultaneously by considering the error as a function of both h and Δt , as shown in Figure 3(c). In this representation the ridge proportional to

$$\Delta t = h^2$$

indicates where the spatial and temporal contributions to the error are comparable. Combinations of $(h, \Delta t)$ below this curve correspond to spatially dominated error with $\text{Error} \sim h^2$, while

combinations above the curve correspond to temporally dominated error with $\text{Error} \sim \Delta t$. The line

$$\Delta t = C_{\max} h$$

represents the stability boundary imposed by the CFL condition; points above this line correspond to unstable approximations with rapidly increasing error.

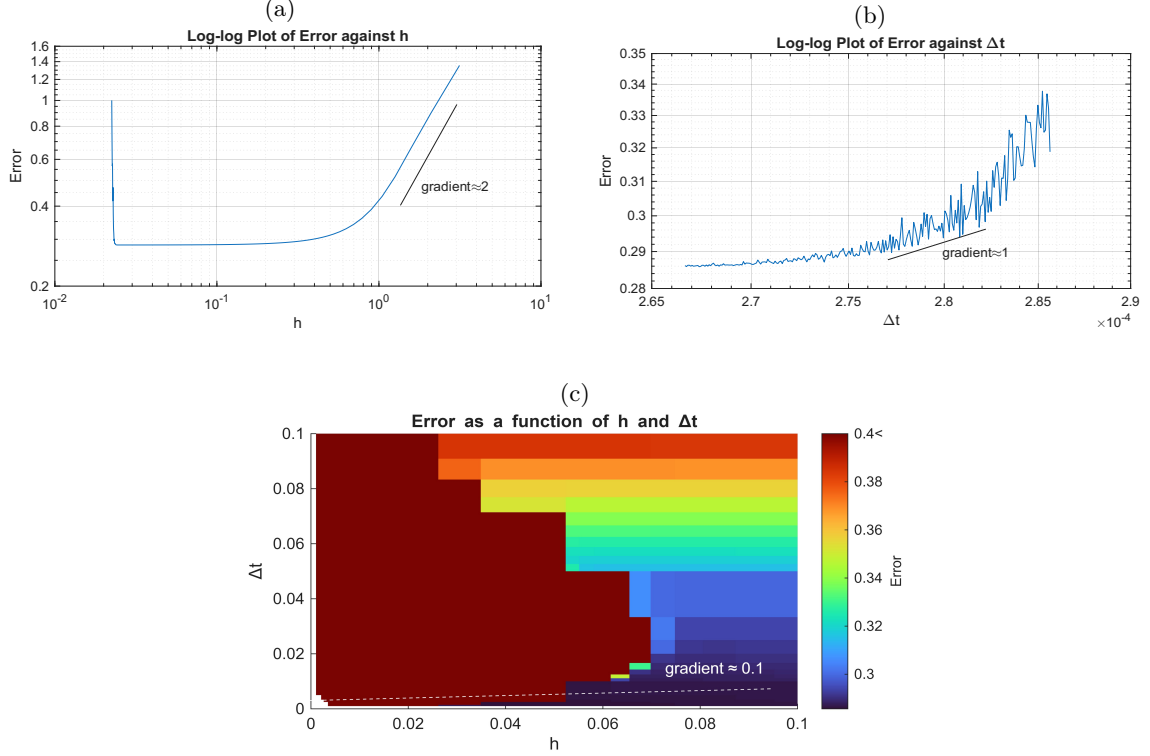


Figure 3: The above is a computation of the error as described in equation (50). (a) Log-log plot of error against h . A section with gradient 2 indicates the dominance of h in the error, the flat section indicates the dominance of Δt and the rapid increase for small h indicates a break in the CFL condition and the consequential instability in the approximation. (b) Log-log plot of error against Δt . A section with gradient 1 indicates the dominance of t in the error, the flat section indicates the dominance of h and the rapid increase for large Δt indicates a break in the CFL condition and the consequential instability in the approximation. (c) Error as a function of h and Δt . The dotted white line of gradient 0.1 indicates $C_{\max}$ with combinations of h and Δt above this line exhibiting a greater chance in unstable approximations.

In practice, the polynomial degree p and the order k of the time stepping scheme should be chosen so that the spatial and temporal discretisations have comparable accuracy. Since a p -th order discontinuous Galerkin method yields spatial error $\mathcal{O}(h^{p+1})$ while a k -th order time integrator produces temporal error $\mathcal{O}(\Delta t^k)$, it is generally advantageous to select $k \geq p+1$ so that the spatial discretisation determines the dominant accuracy. Once p and k have been chosen, the timestep should then be selected to satisfy the CFL condition while ensuring that Δt is sufficiently small that the temporal error does not dominate the spatial error.

6 Example: Discontinuous Advection

A key strength of the discontinuous Galerkin method is its ability to represent solutions containing discontinuities. Such discontinuities may arise directly from the initial data, or they may develop

dynamically during the evolution of the system. Discontinuous initial conditions commonly occur in practical applications where physical quantities change abruptly. Examples include sharp interfaces between fluids of different densities, sudden changes in concentration of a transported chemical species, or step-like temperature distributions in heat transport problems. Alternatively, discontinuities may develop even when the initial data is smooth. This occurs in nonlinear hyperbolic conservation laws where characteristics intersect and form shocks. Typical examples include compressible gas dynamics, traffic flow models, and shallow water equations describing breaking waves.

In order to investigate how discontinuities propagate through time, we consider a simple linear advection problem with a periodic square wave initial condition:

$$\begin{aligned} \frac{\partial u}{\partial t} + \frac{\partial u}{\partial x} &= 0, & 0 \leq x \leq 2, \quad t \geq 0, \\ u(x, 0) &= \begin{cases} 1, & 0 \leq x \leq 1, \\ 0, & \text{otherwise.} \end{cases} \end{aligned} \quad (58)$$

This configuration provides a simple but informative test case. The square wave contains two discontinuities and therefore allows the behaviour of the numerical scheme near sharp interfaces to be examined directly. At the same time, the underlying equation is linear and the exact solution is known explicitly, which makes it straightforward to compare the numerical approximation with the true solution.

As before, the general solution of the linear advection equation given in equation (42) can be used to obtain the analytical solution of system (58):

$$u(x, t) = \begin{cases} 1, & t - 2 \lfloor \frac{t}{2} \rfloor \leq x \leq t + 1 - 2 \lfloor \frac{t+1}{2} \rfloor, \\ 0, & \text{otherwise.} \end{cases} \quad (59)$$

Although this expression may not appear immediately intuitive, it represents a simple translation of the initial square wave to the right with periodic wrapping at the boundaries.

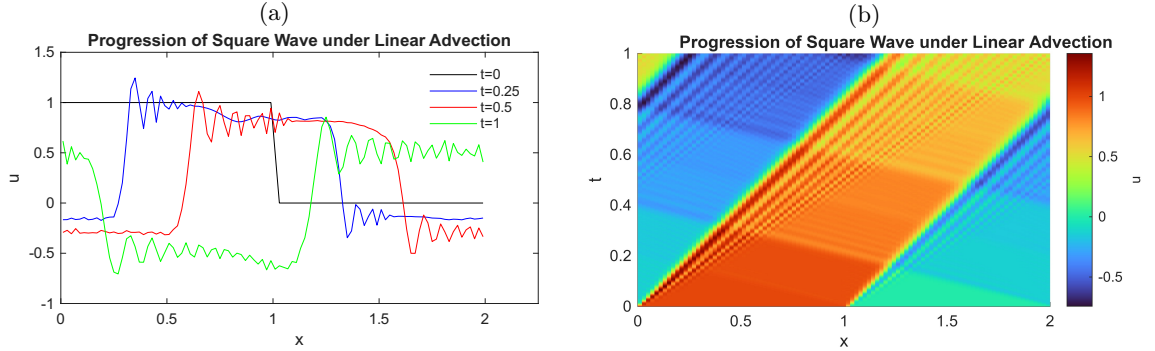


Figure 4: A discontinuous Galerkin approximation to system (58). The Galerkin method used is first order and implements a first order explicit time stepper with spatial and temporal resolutions of 0.02 and 0.001 respectively. (a) and (b) give different perspectives on how the approximation develops through time.

Figure 4 illustrates the discontinuous Galerkin approximation to the solution. The method is first order and uses a first order explicit time stepper. The spatial and temporal resolutions are 0.02 and 0.001 respectively. The approximation broadly captures the translation of the square wave to the right, however several important differences from the exact solution are apparent:

- A gradual decrease in the magnitude of the approximation across all values of x as time advances.
- Oscillations that develop to the right of each discontinuity, which increase in amplitude and propagate in the direction of information travel.

Both of these features represent a significant departure from the true solution, with the approximation losing its characteristic square profile over time. It is important to note that the spatial and temporal resolutions satisfy the CFL condition, and therefore instability arising from an inappropriate choice of timestep can be ruled out as the cause of this deterioration in accuracy. The following section examines which aspects of the numerical method lead to these behaviours in the discontinuous setting and discusses modifications that can be introduced to improve the quality of the approximation.

7 Handling Discontinuities

Discontinuities in physical quantities arise in a wide range of practical problems. They may be imposed directly through the initial conditions or may develop dynamically as the solution evolves. Examples of imposed discontinuities include step-like temperature profiles in heat transfer, abrupt changes in pollutant concentration in environmental transport models, or interfaces separating fluids of different densities. In these situations the discontinuity represents a sharp physical interface that is transported by the flow.

Alternatively, discontinuities may emerge even when the initial data is smooth. In nonlinear hyperbolic conservation laws, characteristics may intersect, causing gradients to steepen until a shock forms. This behaviour occurs in many important applications, including compressible gas dynamics where shock waves develop in supersonic flows, traffic flow models where vehicle density can abruptly increase, and shallow water equations where breaking waves produce hydraulic jumps. In these settings the discontinuity is not prescribed but is generated by the dynamics of the governing equations.

The subsequent behaviour of a discontinuity depends strongly on the underlying physical processes represented by the model. In systems where diffusion or viscosity is present, discontinuities tend to smear over time as the diffusive effects smooth the solution. For example, in heat conduction or viscous fluid flow, sharp gradients gradually broaden as energy or momentum diffuses through the medium. In contrast, purely hyperbolic transport problems contain no such smoothing mechanism, and discontinuities may propagate without changing shape. This occurs, for instance, when a sharp interface between two materials is simply transported by a uniform flow, or when a shock wave travels through an inviscid fluid.

From the perspective of numerical methods, accurately representing such behaviour is challenging. As demonstrated in the previous section, a basic discontinuous Galerkin implementation applied to a discontinuous solution may exhibit oscillations near the discontinuity and a gradual loss of amplitude in the transported signal. These effects are not caused by instability, since the CFL condition is satisfied, but rather arise from the interaction between polynomial approximation and sharp gradients. In particular, polynomial representations tend to produce Gibbs-type oscillations near discontinuities, while certain numerical fluxes and discretisation choices can introduce artificial diffusion that reduces the magnitude of the solution.

Ideally, a numerical scheme should preserve the essential structure of discontinuities while allowing the level of diffusion to reflect the physics of the problem being modelled. In practice, this requires modifications to the basic Galerkin formulation. In the following subsections we will examine several key components of the algorithm that influence the behaviour of discontinuities. First, we consider the role of higher-order time integration and its effect on temporal accuracy. We then introduce limiters, which are designed to suppress spurious oscillations near discontinuities while preserving high-order accuracy in smooth regions. Finally, we discuss the choice of numerical flux, particularly the distinction between upwinding and downwinding, and how this choice influences the amount of numerical diffusion present in the scheme.

7.1 Higher Order Time Steppers

In the discontinuous example considered in Section 6, a first order Galerkin method was combined with a first order time stepper using spatial and temporal resolutions of 0.02 and 0.001 respectively. Despite these resolutions satisfying the CFL condition, the approximation deteriorated noticeably even over the relatively short time interval $T = 1$. This indicates that the quality of

the approximation can be highly sensitive to the temporal discretisation when discontinuities are present.

When it no longer becomes computationally feasible to improve the temporal resolution, upgrading the order of the time stepper becomes the natural approach to further refining the approximation. Recall that improving the order of Galerkin method only improves the accuracy of the approximation in smooth areas away from the discontinuity whereas the accuracy near the discontinuity is capped. As we saw in the square wave advection example, oscillations propagate from the discontinuity and so any smooth areas vanish rapidly. Consequently, even a single discontinuity can eventually influence a large portion of the computational domain.

Figure 5 compares the approximation to system (58) obtained using two different time integration schemes. In both cases the order of the Galerkin method, as well as the spatial and temporal resolutions, are identical; only the order of the time stepper differs. The higher order scheme produces a noticeably more accurate propagation of the square wave, with the characteristic shape of the solution being preserved for a longer period of time. It should also be noted that the fourth order simulation is shown over a longer time interval than the first order simulation, which further emphasises the improvement in accuracy.

Nevertheless, the presence of discontinuities still leads to oscillatory behaviour in the approximation. Even with the higher order time stepper, oscillations develop near the discontinuities and subsequently grow and propagate through the domain. In the first order case these oscillations rapidly dominate the solution and cause the approximation to deteriorate significantly. The higher order time stepper delays this process, but does not eliminate it. This observation indicates that improvements in temporal accuracy alone are insufficient to control the oscillations associated with discontinuities. Additional modifications to the numerical scheme are therefore required, which motivates the introduction of limiters.

7.2 Limiters

It is clear that improving the order of the time stepper alone is not sufficient to produce high quality approximations over long timescales. Instead, the formation of oscillations originating from discontinuities must be addressed directly. In this section we discuss how discontinuities lead to non-physical numerical behaviour, commonly referred to as Gibbs-type oscillations, how such behaviour may be detected, and how limiters such as the generalised minmod limiter can be used to suppress their formation.

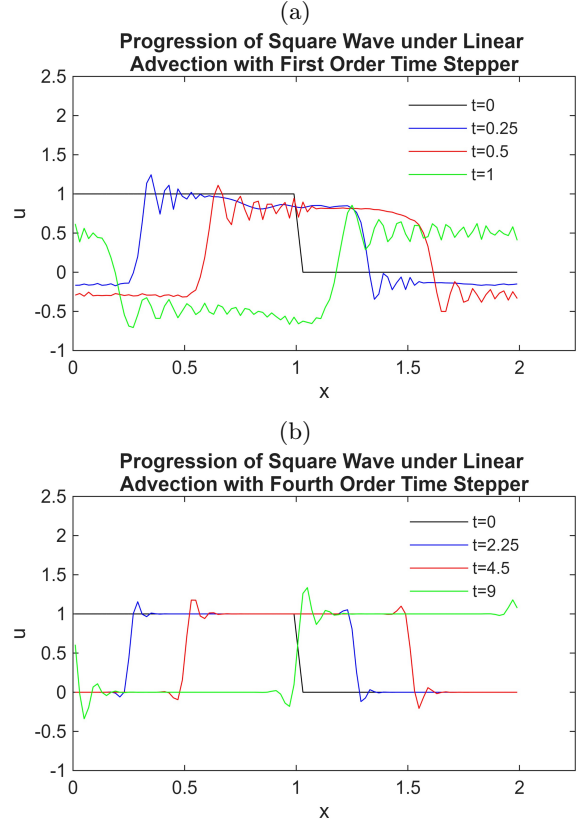


Figure 5: Comparison of Galerkin approximations to the discontinuous system (58) obtained using different time steppers. (a) uses a first order explicit time stepper, whilst (b) uses a fourth order explicit time stepper. Both employ spatial and temporal resolutions of 0.02 and 0.0001 respectively. Note also that the progression of the fourth order system is shown over nine seconds, whereas the first order approximation is shown over one second. This highlights the improved long-time accuracy of the higher order time stepper.

7.2.1 Gibbs-Type Oscillations

To understand the oscillatory behaviour initiated by discontinuities, it is helpful to examine more closely how the degrees of freedom are updated. Using a first order Galerkin method with a first order time stepper as a simple illustrative example, recall that the degrees of freedom evolve according to

$$\dot{\mathbf{U}}^{e,t} = \frac{2}{h} \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix} \begin{pmatrix} -\frac{1}{2} \int_{-1}^1 f(u) ds + h(u_1^{e-1}, u_0^e) \\ \frac{1}{2} \int_{-1}^1 f(u) ds - h(u_1^e, u_0^{e+1}) \end{pmatrix}. \quad (60)$$

Suppose that the internal degrees of freedom in element e are fixed. Discontinuities then arise from the boundary degrees of freedom of neighbouring elements, namely mismatches $u_1^{e-1} \neq u_0^e$ and $u_1^e \neq u_0^{e+1}$. These neighbouring quantities appear in equation (60) only through the numerical flux terms $h(u_1^{e-1}, u_0^e)$ and $-h(u_1^e, u_0^{e+1})$.

For a consistent monotone numerical flux $h(a, b)$ one typically has

$$0 \leq \frac{\partial h(a, b)}{\partial a} \quad \text{and} \quad 0 \geq \frac{\partial h(a, b)}{\partial b}, \quad (61)$$

reflecting the fact that diffusion acts independently of the direction of flow. Considering the influence of neighbouring boundary degrees of freedom on the corresponding degrees of freedom inside the element therefore yields

$$0 \leq \frac{\partial \dot{u}_0^{e,t}}{\partial u_1^{e-1,t}} \quad \text{and} \quad 0 \leq \frac{\partial \dot{u}_1^{e,t}}{\partial u_0^{e+1,t}}. \quad (62)$$

This is consistent with physical intuition: with diffusion, increasing the quantity in a specific location will increase the quantity in neighbouring locations.

If instead we consider the influence of neighbouring boundary degrees of freedom on the opposite degrees of freedom inside the element, we obtain

$$0 \geq \frac{\partial \dot{u}_1^{e,t}}{\partial u_1^{e-1,t}} \quad \text{and} \quad 0 \leq \frac{\partial \dot{u}_0^{e,t}}{\partial u_0^{e+1,t}}. \quad (63)$$

Consequently, changes in neighbouring elements tend to shift the opposite boundary values of an element in the opposite direction. This behaviour is a consequence of the discrete approximation rather than a physical effect. In particular, it reflects the tendency of the numerical scheme to attempt to anticipate sharp changes in the solution. This feature is present for Galerkin methods of any order and for a wide class of physical and numerical flux functions.

To illustrate this mechanism, consider the behaviour of the method in the neighbourhood of a discontinuity in a square wave under linear advection, as shown in Figure 6. For linear advection, the Lax-Friedrichs numerical flux reduces to the upwind flux, so that $h(a, b) = a$. Consequently the scheme detects discontinuities only from the left, and the relevant contribution arises from the term $h(u_1^{e-1}, u_0^e)$.

At time $t = 0$, consider the element immediately to the right of the discontinuity, denoted by e . This element is identical to its right neighbour except for the contribution from the flux term $h(u_1^{e-1}, u_0^e)$. Since $h(u_1^{e-1}, u_0^e) > h(u_1^e, u_0^{e+1})$, the value u_0^e increases in order to match the larger value on the left of the discontinuity, whilst u_1^e decreases due to the coupling described above.

After a single time step, this adjustment produces a new mismatch at the right boundary of the element. Notably, the orientation of this new discontinuity is reversed relative to the original one. In the next iteration the neighbouring element to the right interprets this mismatch in the opposite manner. Repeating this process causes the discontinuity to propagate through successive elements, with the orientation of the mismatch alternating from element to element. The resulting alternating pattern of increases and decreases produces oscillatory behaviour in the numerical solution, which is commonly referred to as Gibbs-type oscillations. These oscillations are purely numerical and do not correspond to any physical feature of the underlying solution.

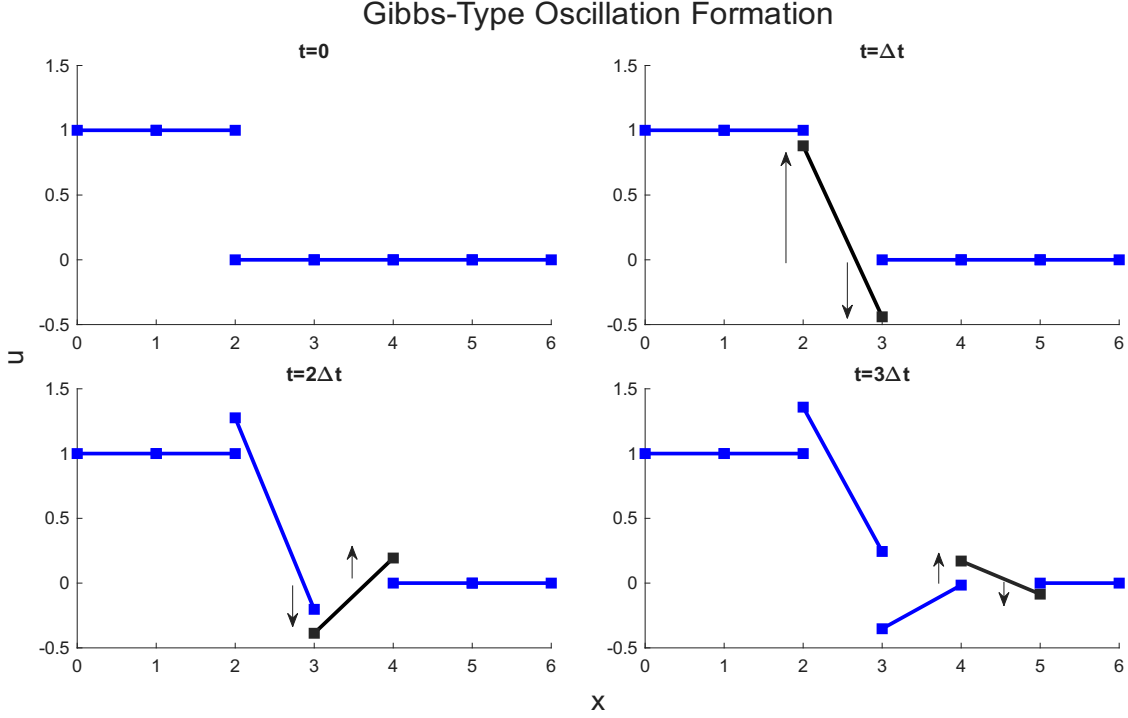


Figure 6: Formation of Gibbs-type oscillations during the progression of a Galerkin approximation to a square wave under linear advection. A linear Galerkin approximation was used with a first order time stepper in order to emphasise the effect. The spatial and temporal resolutions were chosen as 0.18 and 0.04 respectively for the same reason. At $t = 0$, the discontinuity at $x = 2$ causes the left-hand side of the element between $x = 2$ and $x = 3$ to increase whilst the right-hand side decreases. This produces a new discontinuity at $x = 3$. In this way, a single discontinuity can propagate through the domain. Importantly, the directions of change of the left and right sides of each element are opposite, and their orientation alternates between neighbouring elements, leading to the oscillatory behaviour known as Gibbs-type oscillations.

7.2.2 Total Variation (TVD)

Having established how Gibbs-type oscillations arise in discontinuous Galerkin approximations, it is useful to introduce a quantitative measure that allows their development to be monitored. One of the most widely used indicators for this purpose is the total variation (TV) of a discretised function. The total variation measures the cumulative magnitude of changes between neighbouring values and therefore provides an indication of the oscillatory behaviour of an approximation. Schemes that are total variation diminishing (TVD) prevent the growth of oscillations and prohibit unbounded increases in the magnitude of the numerical solution. Consequently, the TVD property plays an important role in controlling the degradation of approximations near discontinuities.

The total variation of a discretised function is defined as follows.

Total Variation (TV): Let u^t be a function discretised spatially by the values $u^{1,t}, u^{2,t}, \dots, u^{n,t}$ at time t . The total variation of the discrete function is defined by

$$TV(u^t) = \sum_{i=1}^{n-1} |u^{i+1,t} - u^{i,t}|. \quad (64)$$

Functions with small total variation are relatively smooth, as neighbouring values vary only gradually. In contrast, functions with large total variation exhibit rapid fluctuations or large jumps between adjacent values. In the context of a discontinuous Galerkin approximation, the quantities $u^{i,t}$ are typically taken to be the cell averages of the numerical solution in each element.

The notion of a total variation diminishing scheme can now be introduced.

Total Variation Diminishing (TVD): Let u^t denote a sequence of discrete approximations indexed by time levels t . The scheme is said to be total variation diminishing if

$$TV(u^{t+1}) \leq TV(u^t) \quad (65)$$

for all discrete time levels t .

A scheme satisfying the TVD property ensures that the total variation of the numerical solution does not increase as the computation progresses. As a result, oscillations cannot grow in magnitude over time. Although some degree of smoothing may occur, this behaviour is generally desirable when approximating discontinuous solutions, since uncontrolled oscillations can rapidly destabilise the approximation.

Within discontinuous Galerkin approximations, the total variation provides a convenient diagnostic tool for detecting Gibbs-type oscillations. As discussed previously, these oscillations originate near discontinuities and propagate through the computational domain as the solution evolves. Their presence is characterised by alternating overshoots and undershoots which significantly increase the total variation of the numerical solution. Consequently, schemes designed to control Gibbs-type oscillations are often constructed so that the resulting approximation satisfies the TVD property, or at least maintains the total variation within a controlled range.

Harten showed that schemes of this type can be guaranteed to satisfy the TVD property provided that suitable bounds are imposed on the boundary values of each element. In the present context, this result can be stated as follows.

Theorem 1. *Consider a linear discontinuous Galerkin method with a monotone numerical flux function and a forward Euler time discretisation. If the boundary values in each element satisfy*

$$\begin{aligned} \min(u^{i-1,t}, u^{i,t}) &\leq u_0^{i,t} \leq \max(u^{i-1,t}, u^{i,t}), \\ \min(u^{i,t}, u^{i+1,t}) &\leq u_1^{i,t} \leq \max(u^{i,t}, u^{i+1,t}), \end{aligned}$$

for all elements i , then the resulting discontinuous Galerkin approximation is total variation diminishing.

In other words, the theorem states that the approximation is TVD if the reconstructed boundary values within each element remain bounded by the neighbouring cell averages. The left boundary value of an element must lie between the cell average of the element and that of its left neighbour, whilst the right boundary value must lie between the cell average of the element and that of its right neighbour. These conditions prevent the polynomial representation within an element from producing new extrema relative to neighbouring values.

The result above was originally established for a forward Euler time discretisation. However, Shu and Osher later showed that the same property holds for certain Runge–Kutta time-stepping schemes, since these schemes can be expressed as convex combinations of forward Euler steps.

The two bounds in Theorem 1 can be expressed more compactly by considering the slope of the approximation within each element.

Theorem 2. *Let*

$$s_i^t = u_1^{i,t} - u_0^{i,t}, \quad \Delta_i^t = u^{i,t} - u^{i-1,t}, \quad \Delta_{i+1}^t = u^{i+1,t} - u^{i,t}.$$

For a linear discontinuous Galerkin method with a monotone numerical flux function, if

$$|s_i^t| \leq 2 \min(|\Delta_i^t|, |\Delta_{i+1}^t|)$$

for all elements i , then the discontinuous Galerkin approximation is TVD.

Proof. Consider the first condition of Theorem 1,

$$\min(u^{i-1,t}, u^{i,t}) \leq u_0^{i,t} \leq \max(u^{i-1,t}, u^{i,t}).$$

For a linear approximation within the element, the left boundary value may be written as

$$u_0^{i,t} = u^{i,t} - \frac{s_i^t}{2}.$$

Substituting this expression into the inequality yields

$$\min(u^{i-1,t}, u^{i,t}) \leq u^{i,t} - \frac{s_i^t}{2} \leq \max(u^{i-1,t}, u^{i,t}).$$

Rearranging gives

$$\min(-\Delta_i^t, 0) \leq -\frac{s_i^t}{2} \leq \max(-\Delta_i^t, 0).$$

Noting that the LHS is strictly non-positive and the RHS is strictly non-negative, we can then take absolute values

$$\begin{aligned} |s_i| &\leq 2 \min(|\min(-\Delta_i^t, 0)|, |\max(-\Delta_i^t, 0)|) = 2 \min(|\max(\Delta_i^t, 0)|, |\max(-\Delta_i^t, 0)|) \\ &\iff |s_i| \leq 2|\Delta_i^t|. \end{aligned}$$

Applying the same argument to the second condition of Theorem 1, and using the relation

$$u_1^{i,t} = u^{i,t} + \frac{s_i^t}{2},$$

gives

$$|s_i^t| \leq 2|\Delta_{i+1}^t|.$$

Combining the two bounds yields

$$|s_i^t| \leq 2 \min(|\Delta_i^t|, |\Delta_{i+1}^t|),$$

which establishes the stated via Theorem 1. $\square$

The implication of the theorem is that discontinuous galerkin schemes are TVD provided the scaled magnitude of the internal slope of an element doesn't exceed those formed with neighbouring elements.

The classical TVD framework is inherently restricted to first-order or piecewise linear approximations. For higher-order discontinuous Galerkin methods, strict TVD conditions are incompatible with high-order accuracy, and are therefore replaced by relaxed criteria such as total variation bounded (TVB) conditions together with nonlinear limiting strategies. Whilst this may appear to limit the applicability of TVD theory, it is important to note that the presence of discontinuities restricts the achievable order of convergence regardless of the polynomial degree used within each element. In particular, the local accuracy in the vicinity of a discontinuity is not improved by increasing the order of the basis functions.

However, higher-order schemes still provide significant advantages away from discontinuities, where the solution remains smooth, and generally yield sharper and less diffusive approximations overall. Consequently, rather than reverting entirely to first order schemes, practical implementations typically employ higher-order Galerkin methods in conjunction with limiters to control oscillations near discontinuities whilst retaining high accuracy elsewhere.

7.2.3 Generalised Minmod Limiters

Limiters are modifications applied to discontinuous Galerkin schemes in order to stabilise the numerical approximation and preserve key qualitative features of the solution over extended time intervals. Since a primary source of instability is the formation of Gibbs-type oscillations near discontinuities, an effective limiter should suppress the growth of oscillations whilst leaving smooth regions largely unaffected. There exist many such limiters, each constructed in a different manner and possessing distinct properties. In this section, we consider the family of generalised minmod limiters, which are straightforward to implement and, when appropriately calibrated, satisfy the TVD property.

Generalised minmod limiters act by enforcing the TVD condition through modification of the internal slope within each element, as characterised in Theorem 2. Whilst this formulation is naturally expressed in the context of first order (piecewise linear) schemes, considering limiters on first order schemes is the natural next step as in the absence of limiters, approximation accuracy is dominated by discontinuities, not the order of the scheme. Beyond this, it serves as a useful illustrative example of how limiting strategies control oscillatory behaviour arising from discontinuities.

The generalised minmod limiter is defined as follows.

Generalised minmod limiter Let $\theta > 0$ be a smoothness parameter and let s_i^t , Δ_i^t and Δ_{i+1}^t be defined as in the previous section. Then,

$$\hat{s}_i^t = \begin{cases} \min(\theta\Delta_i^t, s_i^t, \theta\Delta_{i+1}^t) & \text{if } s_i^t, \Delta_i^t \text{ and } \Delta_{i+1}^t \text{ are all positive,} \\ \max(\theta\Delta_i^t, s_i^t, \theta\Delta_{i+1}^t) & \text{if } s_i^t, \Delta_i^t \text{ and } \Delta_{i+1}^t \text{ are all negative,} \\ 0 & \text{otherwise.} \end{cases}$$

The degrees of freedom for element i are then updated via

$$u_0^{i,t} = u^{i,t} - \frac{1}{2}\hat{s}_i^t, \quad u_1^{i,t} = u^{i,t} + \frac{1}{2}\hat{s}_i^t.$$

A direct consequence of this definition is that, provided s_i^t , Δ_i^t and Δ_{i+1}^t share the same sign,

$$\text{if } |s_i^t| < \theta \min(|\Delta_i^t|, |\Delta_{i+1}^t|), \quad \text{then } \hat{s}_i^t = s_i^t. \quad (66)$$

Hence, when the internal slope is sufficiently small relative to neighbouring slopes, the limiter leaves the approximation unchanged. This ensures that smooth regions are preserved, which is a fundamental requirement for any effective limiting strategy.

The parameter θ controls the amount of limiting applied. Smaller values of θ result in more aggressive limiting, reducing the magnitude of the internal slope and thereby increasing numerical diffusion. Larger values of θ allow steeper gradients to persist. The classical minmod limiter corresponds to $\theta = 1$, whilst $\theta = 2$ yields the monotonised central (MC) limiter. In practice, values $1 \leq \theta \leq 2$ are commonly employed, providing a balance between suppressing oscillations and avoiding excessive smoothing. This parameter may also be interpreted as modelling different physical regimes, with smaller values corresponding to more diffusive systems and larger values representing systems with sharper features.

The generalised minmod limiter can be shown to enforce the TVD condition under suitable constraints. Indeed,

$$|\hat{s}_i^t| \leq \min(|\theta\Delta_i^t|, |s_i^t|, |\theta\Delta_{i+1}^t|) \leq \theta \min(|\Delta_i^t|, |\Delta_{i+1}^t|). \quad (67)$$

Therefore, by Theorem 2, if $\theta \leq 2$, the resulting scheme is TVD. This identifies $\theta = 2$ as the largest value for which the limiter guarantees the TVD property whilst minimising artificial diffusion.

The structure of the limiter reveals that modifications are only applied when the internal slope is inconsistent with neighbouring gradients or when sign changes occur. In particular, when s_i^t , Δ_i^t and Δ_{i+1}^t do not share a common sign, the slope is set to zero. This mechanism is crucial for suppressing Gibbs-type oscillations, which are characterised by alternating slopes near discontinuities. By eliminating such alternating behaviour, the limiter prevents the propagation of spurious oscillations through the domain, facilitating greater stability than simply enforcing TVD.

Since the TVD conditions need to be met before a time step, generalised minmod limiters are applied prior to the evaluation of temporal derivatives. For multi-stage time stepping schemes such as Runge-Kutta methods, the limiter must be applied at each intermediate stage to ensure stability. For example, in a fourth order Runge-Kutta scheme, the limiter is applied before the computation of each stage derivative, k_i .

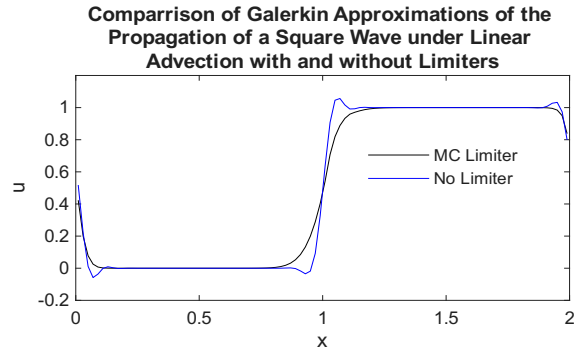


Figure 7: Comparison of Galerkin approximations to the discontinuous system (58) with and without limiters. In both cases the profile is taken at $T = 1$, a first order Galerkin scheme is used with an explicit RK4 timestepper and the spatial and temporal resolutions are 0.02 and 0.0001 respectively. The blue line does not employ a limiter whereas the black line uses an MC limiter (a generalised minmod limiter with $\theta = 2$).

Figure 7 demonstrates the effect of applying a generalised minmod limiter. The limited solution suppresses oscillations near discontinuities whilst preserving the structure of the solution in smooth regions, leading to a more stable approximation over longer times.

Despite their simplicity and robustness, minmod limiters possess certain limitations. In particular, they introduce a significant amount of numerical diffusion, even when $\theta = 2$. Moreover, the TVD framework upon which they are based is inherently restrictive and most naturally suited to low-order approximations.

Alternative measures of oscillatory behaviour include, for example, monitoring local extrema, enforcing maximum principles, or controlling higher-order moments of the solution. Correspondingly, more sophisticated limiters have been developed. These include TVB limiters, which relax the strict TVD condition by allowing limited growth in total variation in smooth regions, and WENO-type reconstructions, which adaptively combine local polynomials to avoid oscillations. Additionally, hierarchical or modal limiters act directly on higher-order coefficients in polynomial expansions, enabling their use in higher-order discontinuous Galerkin methods.

These approaches retain the ability to capture sharp features whilst reducing excessive diffusion, and form the basis of modern high-order discontinuous Galerkin implementations for problems involving discontinuities.

7.3 Diffusivity in Numerical Fluxes

We have seen how higher order time steppers, when combined with limiters, can be used to approximate discontinuities in a manner that preserves sharp features whilst suppressing non-physical oscillations. A further, more subtle, consideration is the rate at which discontinuities are expected to smooth. Whilst limiters introduce stability at the expense of additional diffusion, often controlled by parameters such as θ in generalised minmod limiters, they are not the sole mechanism by which smoothness is regulated. Numerical fluxes also play a fundamental role in controlling diffusivity. In practice, both the choice of limiter and numerical flux must be considered together in order to represent different physical diffusion regimes accurately.

The Lax-Friedrichs numerical flux can be generalised as

$$h(a, b) = \underbrace{\frac{1}{2} (f(a) + f(b))}_{\text{central flux}} - \underbrace{\frac{\alpha}{2} \max_{a \leq \zeta \leq b} |f'(\zeta)| (b - a)}_{\text{artificial viscosity}}, \quad (68)$$

where the second term introduces artificial viscosity across element interfaces. The magnitude of this diffusive contribution is controlled by the parameter α . If $\alpha < 1$, the dissipation may be insufficient relative to the maximum wave speed $|f'(u)|$, and unstable oscillations may develop. Conversely, if $\alpha \gg 1$, excessive artificial diffusion is introduced, leading to significant smearing of sharp features. In this way, the generalised Lax-Friedrichs flux provides a direct mechanism for tuning diffusivity.

In the context of advective systems, the evolution of the solution is governed by the direction of information propagation. This motivates the purely upwinding numerical flux,

$$h(a, b) = \begin{cases} f(a), & \text{if } f'(u) > 0, \\ f(b), & \text{if } f'(u) < 0, \end{cases} \quad (69)$$

which selects the upwind state in accordance with the characteristic direction. This choice ensures that information is transported in a physically consistent manner across element interfaces and introduces only the minimal amount of numerical diffusion required for stability.

For linear advective fluxes, the purely upwinding numerical flux coincides with the Lax-Friedrichs flux with $\alpha = 1$. Since $f'(u) = C$ for all u , we obtain

$$h(a, b) = \frac{1}{2} [Ca + Cb - |C|(b - a)] = \frac{C}{2} [a + b - \text{sgn}(C)(b - a)] = \begin{cases} Ca, & \text{if } C > 0, \\ Cb, & \text{if } C < 0, \end{cases} \quad (70)$$

which is precisely the upwinding flux.

For nonlinear flux functions, however, the situation differs. The Lax-Friedrichs flux typically employs a global or local upper bound on $|f'(u)|$, which may overestimate the true wave speed and therefore introduce excessive diffusion. In contrast, upwinding fluxes based on local characteristic information can achieve stability with reduced diffusion, thereby preserving sharper features. As with limiters, the choice of numerical flux must balance stability against the desire to minimise artificial smoothing. In problems where sharp features are expected to persist over long times, this balance becomes particularly delicate.

In such cases, more sophisticated fluxes may be employed. For example, the Harten-Lax-van Leer (HLL) flux approximates the solution of the Riemann problem at each interface using estimates of the relevant wave speeds. By more accurately capturing the propagation of information, HLL-type fluxes can reduce unnecessary diffusion compared to Lax-Friedrichs whilst maintaining stability. These fluxes are therefore well suited to problems in which sharp discontinuities must be resolved with minimal smearing.

8 Example: Continuous Inviscid Burger's Equation

We now turn our attention to examples involving non-linear fluxes. The inviscid Burger's equation possesses a simple non-linear flux operator and serves as a prototype for conservation laws that can develop discontinuities. To motivate the form of the inviscid Burger's equation, recall that the general form of the convective flux is given by

$$\mathbf{j}_{\text{conv}} = u\mathbf{v} = f(u),$$

where $\mathbf{v}$ is the velocity of the surrounding medium. In many practical contexts, the velocity depends on the density of the quantity itself, that is $\mathbf{v} = \mathbf{v}(u)$. For example, in traffic flow, the speed of vehicles depends on the local density of cars, with freer roads allowing faster motion. Similar behaviour arises in crowd dynamics within confined spaces and in granular flow through chutes.

The simplest way to model this dependence is through a linear relationship, $\mathbf{v} \propto u$, which leads to a quadratic flux of the form

$$f(u) = Cu^2 \tag{71}$$

for some constant C . Choosing $C = \frac{1}{2}$ yields a form consistent with a simplified version of the Euler momentum equation under the assumptions of uniform density and negligible pressure gradients and external forces. In this case, $\mathbf{v} = \frac{u}{2}$, so regions of higher density propagate more rapidly. This behaviour is representative of nonlinear acoustics, where regions of higher pressure travel faster. Alternatively, it can model the relationship between fluid height and wave speed. Shallower fluid experiences higher degrees of relative friction from the solid bottom and so travels slower than deeper fluid. In such situations, faster moving regions catch up with slower ones, leading to the steepening of gradients and eventual wave breaking in the absence of sufficient diffusion.

The Burger's equation in one spatial dimension is given by

$$\frac{\partial u}{\partial t} + \frac{\partial \left(\frac{u^2}{2} \right)}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}, \tag{72}$$

where the term on the right-hand side represents diffusion. This diffusive contribution smooths the solution and prevents the formation of infinite gradients. In regimes where ν is negligible, the equation reduces to the inviscid Burger's equation:

$$\frac{\partial u}{\partial t} + \frac{\partial \left(\frac{u^2}{2} \right)}{\partial x} = 0. \tag{73}$$

The method of characteristics yields an implicit solution of the form

$$u(x, t) = u_0(x - ut), \tag{74}$$

valid while characteristics do not intersect. This general solution can be validated easily by finding the temporal and spatial derivatives in turn. Let $w = x - ut$, then by applying the chain rule w.r.t. t :

$$\frac{\partial u}{\partial t} = \frac{\partial u}{\partial w} \frac{\partial w}{\partial t} = \frac{\partial u}{\partial w} \left(-t \frac{\partial u}{\partial t} - u \right). \quad (75)$$

Rearranging for the temporal derivative gives:

$$\frac{\partial u}{\partial t} = \frac{[-u \frac{\partial u}{\partial w}]}{[1 + t \frac{\partial u}{\partial w}]}. \quad (76)$$

Applying consecutive chain rules w.r.t. x gives:

$$\frac{\partial \left(\frac{u^2}{2} \right)}{\partial x} = \frac{\partial \left(\frac{u^2}{2} \right)}{\partial u} \frac{\partial u}{\partial w} \frac{\partial w}{\partial x} = u \frac{\partial u}{\partial w} \left(1 - t \frac{\partial u}{\partial x} \right) = u \frac{\partial u}{\partial w} \left(1 - \frac{t}{u} \frac{\partial \left(\frac{u^2}{2} \right)}{\partial x} \right). \quad (77)$$

Rearranging for the spatial derivative then gives,

$$\frac{\partial \left(\frac{u^2}{2} \right)}{\partial x} = \frac{[u \frac{\partial u}{\partial w}]}{[1 + t \frac{\partial u}{\partial w}]}, \quad (78)$$

which satisfies the inviscid Burger's equation. Although equation (87) appears simple, it is implicit in u , and the explicit inversion is generally not possible to derive for non-algebraic initial data. Consequently, qualitative analysis is often more informative than attempting to obtain closed-form solutions.

Rewriting the flux derivative using the chain rule gives

$$\frac{\partial \left(\frac{u^2}{2} \right)}{\partial x} = u \frac{\partial u}{\partial x}, \quad (79)$$

so that the equation can be expressed as

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = 0. \quad (80)$$

This form provides useful physical insight. For a given location, the change in the density is proportional to the value of the density here and the direction of the change is controlled by the spatial gradient (provided the density takes positive values only- this reflects almost all practical contexts). Positive spatial derivatives result in a decrease in the density whereas negative spatial derivatives result in an increase in the density. As a result, for sections of positive spatial gradient, larger densities are decreased more resulting in a profile with flattened positive spatial derivatives. On the other hand, for sections with negative spatial derivatives, larger densities are increased more resulting in a profile with steeper negative spatial derivatives. Furthermore, for a given location, the change in density is proportional to spatial derivative, which means steeper positive slopes get flattened more and steeper negative slopes are steepened more.

The time at which the solution ceases to remain smooth is known as the breaking time, given by

$$t_b = -\frac{1}{\min_{x \in X} u'_0(x)}, \quad (81)$$

provided that $\min_{x \in X} u'_0(x) < 0$. This expression shows that shock formation is entirely determined by the initial condition. Whilst not all initial conditions lead to shock formation, many physically relevant ones do.

We now consider the propagation of a sinusoidal wave under inviscid Burger's flow:

$$\left. \begin{aligned} \frac{\partial u}{\partial t} + \frac{\partial \left(\frac{u^2}{2} \right)}{\partial x} &= 0, \\ u(x, 0) &= 1.5 + \sin(x), \end{aligned} \right\} \quad 0 \leq x \leq 2\pi, \quad t \geq 0. \quad (82)$$

The sinusoidal initial condition represents a smooth, naturally occurring wave profile. In acoustics, such a profile may arise from a vibrating source, whilst in fluid systems it may represent surface waves generated by wind. The variation in amplitude allows faster moving regions to overtake slower ones, leading to progressive distortion of the wave.

The implicit solution is given by

$$u = 1.5 + \sin(x - ut), \quad (83)$$

for which there is no closed form inversion due to the non algebraic sin function. Unlike the exponential or logarithmic function, a Lambert W type function can't be used in this context either. Furthermore, $u'_0(x) = \cos(x)$ and so $\min_{x \in \mathbf{X}} u'_0(x) = -1$ meaning we expect the discontinuity to occur at $t = 1$.

Periodic boundary conditions are imposed so that information leaving one boundary re-enters at the opposite boundary. The Galerkin approximation is shown in Figure 8 at times $t = 0$, $t = 0.5$, $t = 1$ and $t = 2$. As predicted, regions with positive gradient flatten, whilst those with negative gradient steepen. Around $t = 1$, the solution approaches the onset of a shock, consistent with the theoretical breaking time.

At later times, such as $t = 2$, the numerical solution exhibits a profile consisting of a relatively gradual increase followed by a sharp drop. In the absence of numerical stabilisation, one would expect the formation

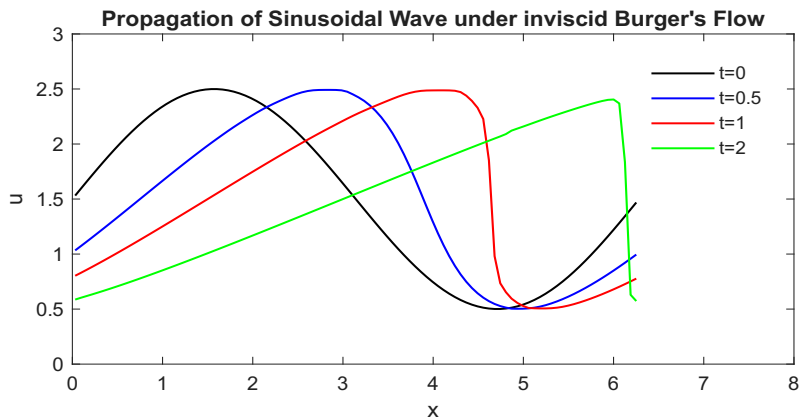


Figure 8: Propagation of the Galerkin approximation to system (82). The Galerkin scheme was linear and used a fourth order time stepper with a LF numerical flux function and an MC limiter. The spatial and temporal resolutions used were 0.02 and 0.001 respectively.

of strong oscillations and a breakdown of the approximation beyond the breaking time. However, the use of an MC limiter enforces a form of nonlinear stability by constraining the solution within admissible bounds. This has the effect of suppressing oscillations and preventing the formation of non-physical extrema.

As a consequence, the discontinuity is not represented as an abrupt jump but rather as a smeared transition region. This smoothing is a direct result of the artificial diffusion introduced by both the limiter and the numerical flux. Whilst this diffusive effect slightly degrades the sharpness of the solution, it enables the scheme to continue producing stable and physically meaningful approximations beyond the breaking time, where the classical solution no longer exists.

9 Example: Discontinuous Inviscid Burger's Equation

In practice, discontinuities often arise from imposed conditions rather than from the intrinsic evolution of the system. Therefore, having examined how non-linear fluxes can generate shocks from smooth initial data, it is also important to understand how such fluxes propagate solutions in which discontinuities are already present. Whilst physical quantities do not exhibit exact discontinuities, modelling sharp transitions as discontinuous greatly simplifies both the mathematical analysis and numerical treatment.

Such situations arise in a variety of contexts. In traffic flow, discontinuities may represent abrupt changes in vehicle density caused by traffic lights, toll booths, or variations in road conditions. In crowd dynamics, they may correspond to sudden changes in density near doorways or constrictions. In granular flow, they may arise from obstacles within a chute. In each case, the discontinuity represents a rapid transition between two distinct states of the system.

We consider the inviscid Burger's equation with a periodic square wave initial condition,

$$\begin{aligned} \frac{\partial u}{\partial t} + \frac{\partial \left(\frac{u^2}{2} \right)}{\partial x} &= 0, & 0 \leq x \leq 2, & t \geq 0, \\ u(x, 0) &= \begin{cases} 1, & 0 \leq x \leq 1, \\ 0, & \text{otherwise.} \end{cases} \end{aligned} \quad (84)$$

This configuration represents a discontinuous transition between two constant states. In acoustics, such a profile may idealise a sharp interface between regions of differing pressure, whilst in shallow water it may approximate a sudden change in fluid depth, for example due to a step in the bed profile.

For discontinuous initial data, the classical solution obtained via the method of characteristics is no longer valid globally, as characteristics intersect immediately at points of discontinuity. Instead, the appropriate framework is that of weak solutions, supplemented by entropy conditions to ensure physical admissibility. In this setting, discontinuities propagate as shocks whose speed is determined by the Rankine-Hugoniot condition,

$$s = \frac{f(u_R) - f(u_L)}{u_R - u_L},$$

where u_L and u_R are the densities to the left and right of the discontinuities respectively. For a Burger's flow,

$$s = \frac{u_R + u_L}{2}, \quad (85)$$

the average density of the discontinuity. The interpretation of this is that the speed at which a discontinuity propagates for Burger's flow isn't dependent on the size of the discontinuity, but rather its average value. For the present system, this yields $s = \frac{1}{2}$ for a jump from $u_L = 1$ to $u_R = 0$, so that the discontinuity propagates with constant speed. The implicit characteristic relation $u(x, t) = u_0(x - ut)$ is therefore not directly applicable across the discontinuity, although it remains valid within each constant state away from the shock.

The notion of a breaking time is also altered in this context. For smooth initial data, the breaking time corresponds to the first intersection of characteristics. However, for discontinuous initial conditions, characteristics intersect immediately at the discontinuity, so the breaking time is effectively $t_b = 0$. As a result, the system must be interpreted in the weak sense from the outset. In numerical schemes incorporating limiters and diffusive numerical fluxes, this immediate shock formation is regularised, and the discontinuity is instead represented by a narrow transition layer whose thickness depends on the numerical diffusion and the flux.

In the Galerkin approximation of system (84), numerical fluxes and limiters introduce diffusive effects which act to smooth sharp transitions. The method therefore represents discontinuities as steep but continuous gradients. The behaviour of these gradients is governed by the non-linear structure of the flux. As established previously, positive spatial gradients tend to flatten, whilst negative spatial gradients tend to steepen.

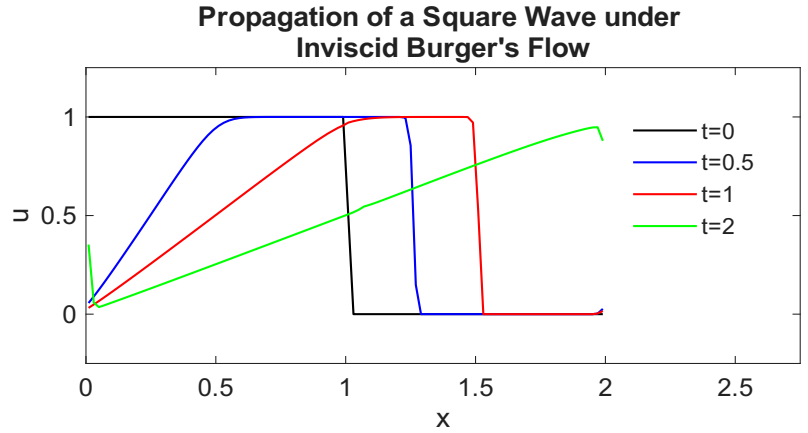


Figure 9: Propagation of the Galerkin approximation to system (84). The Galerkin scheme was linear and used a fourth order time stepper with a LF numerical flux function and an MC limiter. The spatial and temporal resolutions used were 0.02 and 0.001 respectively.

In the present configuration, the left interface of the square wave corresponds to a jump from 0 to 1, which forms a positive gradient. This transition therefore spreads out over time, producing a rarefaction-like structure. In contrast, the right interface corresponds to a jump from 1 to 0, which forms a negative gradient. This transition remains sharp and propagates as a shock, with its steepening tendency counteracted only by the numerical diffusion introduced by the flux and limiter.

The flat regions of the square wave are also affected asymmetrically. The portion adjacent to the positive jump develops a gradual slope due to the spreading of the rarefaction, whilst the portion adjacent to the negative jump remains relatively sharp. As a result, the overall profile evolves into a trapezoidal shape, with a widening lower plateau and a narrowing upper plateau.

The numerical approximation shown in Figure 9 reflects this behaviour. The smoothing of the positive jump and the persistence of the negative jump are clearly visible, demonstrating how the non-linear flux distinguishes between the two orientations of discontinuity. The shocks in the approximation also travel at a speed matched by the theoretical one derived from the Rankine-Hugoniot condition. This example highlights a key feature of inviscid Burger’s dynamics: the evolution of discontinuities is strongly dependent on their orientation, with expansion fans forming from upward jumps and shocks forming from downward jumps.

10 Conclusion

The code used to implement the discontinuous Galerkin method, as well as to generate the data in the figures, is provided in Appendix 11.1, with the vector and matrix header files included in Appendices 11.2 and 11.3 respectively. Note that, in order to interchange different flux functions, the elements must be stored as pointers so that derived classes can be invoked appropriately.

Convection-dominated problems encompass a wide range of physical scenarios. Whilst analytical approaches are the gold standard for representing solutions, numerical methods are required in more complex cases, such as those involving non-linear fluxes or discontinuities (a simplification of shocks in a system). The discontinuous Galerkin method provides an adaptable framework capable of handling these complications. In comparison to more sophisticated numerical approaches, the discontinuous Galerkin method may exhibit reduced accuracy in highly complex settings. Despite this, its parallelisability, arising from its relatively local structure, makes it computationally efficient in many practical situations, preserving its role as a benchmark among mid-tier numerical methods.

The project begins by outlining a derivation of the transport equation, demonstrating that it arises directly from conservation laws. By considering the weak form of the solution and polynomial basis functions over a discretised domain, the Galerkin approximation is formulated. This approximation consists of the mass matrix (which must be inverted to propagate the solution), the flux vector, and the numerical flux vector, all defined at the level of a single element.

When increasing the degree of the polynomial approximation within each element, the only essential requirement is that the chosen basis functions span $\{1, x, \dots, x^n\}$. Whilst, in principle, many choices are possible, certain bases significantly simplify implementation. In this regard, the inversion of the mass matrix and the evaluation of the flux vector are the principal computational bottlenecks. The use of Legendre polynomials yields a diagonal mass matrix by construction, thereby trivialising its inversion, although flux vector evaluation remains relatively costly. Alternatively, Lagrange polynomials constructed using Gauss-Lobatto nodes, together with consistent quadrature for both the mass matrix and flux evaluation, lead to an approximate diagonal mass matrix and a significantly simplified flux computation. The use of higher-order time-stepping schemes, such as Runge-Kutta methods, requires careful implementation, as intermediate stages must be computed consistently. Implicit time-stepping schemes offer enhanced stability but are computationally more demanding and are typically restricted to linear problems.

The effectiveness of the discontinuous Galerkin method depends on appropriate configuration. To assess this, an error analysis is required. Numerical stability is governed by the Courant–Friedrichs–Lewy condition,

$$C = \frac{\max_{a \leq \zeta \leq b} |f'(\zeta)| \Delta t}{h} \leq C_{\max},$$

where Δt and h denote the temporal and spatial resolutions respectively. Violation of this condition leads to exponential growth of the error in time. When stability is ensured,

$$\text{Error} \sim \mathcal{O}(h^{p+1}) + \mathcal{O}(\Delta t^k),$$

where p is the polynomial degree and k is the order of the time stepper. Increasing p or introducing limiters typically reduces the admissible CFL constant $C_{\max}$, whilst increasing k generally relaxes this restriction. In the presence of discontinuities, the spatial error degrades to $\mathcal{O}(h^{1/2})$ near the discontinuity, independently of p , whilst remaining $\mathcal{O}(h^{p+1})$ in smooth regions. This reduced-order error propagates throughout the domain, reflecting the dominant influence of the discontinuity. The use of limiters typically improves the local order near discontinuities to $\mathcal{O}(h)$, although they may reduce accuracy in smooth regions. Crucially, limiters suppress the rapid spread of error, resulting in improved global accuracy. In practice, choosing $k = p + 1$ together with a conservative timestep such as $\Delta t = \frac{0.05h}{\max|f'(\zeta)|}$ (or smaller) provides a robust balance between stability and efficiency.

The behaviour of the method in the presence of discontinuities is illustrated using a periodic square wave under linear advection. Without a limiter, the approximation rapidly destabilises due to Gibbs-type oscillations. These arise from the interaction of boundary degrees of freedom, which generate alternating discontinuities that propagate through the domain. This behaviour reflects the structure of the numerical scheme rather than any underlying physical phenomenon. Refining spatial and temporal resolutions or increasing the polynomial degree and order of time stepper delays, but does not eliminate, these oscillations. This motivates the introduction of limiters. Among the available options, generalised minmod limiters provide a simple and effective mechanism by enforcing a total variation diminishing property through the adjustment of internal elemental slopes. This suppresses the formation of non-physical oscillations whilst preserving smooth regions. The incorporation of limiters within higher-order Runge-Kutta schemes requires their application at each intermediate stage. The numerical flux governs communication between elements; the generalised Lax-Friedrichs family offers a robust choice with controllable artificial viscosity. The parameters θ (limiter) and α (flux) regulate the degree of stabilisation with profiles that smoothen faster arising from implementations with more stability. In this way, varying degrees of diffusion can be introduced implicitly which facilitates the representation of regimes with different diffusive rates..

The method is then applied to non-linear fluxes, in particular the inviscid Burgers' equation. The evolution of a periodic sinusoidal profile exhibits the expected flattening of positive gradients and steepening of negative gradients, culminating in shock formation at the theoretically predicted breaking time. Beyond this point, classical solutions cease to exist, but the discontinuous Galerkin method continues to produce physically meaningful approximations, a flattening profile with a discontinuity propagating forward. The limiter suppresses oscillations and prevents the formation of spurious extrema, while introducing mild diffusion. When applied to a square wave initial condition, the method captures the asymmetric behaviour of discontinuities: upward jumps develop into rarefaction-like structures, whilst downward jumps persist as shocks. The propagation speeds of these discontinuities agree with those predicted by the Rankine-Hugoniot condition.

The discontinuous Galerkin method remains a powerful tool for approximating solutions to convection-dominated problems where analytical approaches are no longer viable. It is capable of capturing non-linear effects and shock dynamics with considerable flexibility. Although more sophisticated methods may offer higher accuracy in extreme regimes, the balance between accuracy, adaptability, and computational efficiency ensures that the discontinuous Galerkin method remains an important and widely used approach in the numerical modelling of transport phenomena.

11 Appendix

11.1

```

1 #include <iostream>
2 #include "mvector.h"
3 #include "mmatrix.h"
4 #include <cmath>
```

```

5  #include <vector>
6  #include <algorithm>
7  #include <string>
8  #include <fstream>
9
10 // advection element class
11 class AdvectionElement
12 {
13     public:
14         // Pointer to the left neighbour
15         AdvectionElement *Left_neighbour_pt;
16
17         // Pointer to the right neighbour
18         AdvectionElement *Right_neighbour_pt;
19
20         // Storage for the coordinates
21         MVector X;
22
23         // Storage for the unknowns
24         MVector U;
25
26         MVector Utemp;
27
28         MVector K1;
29
30         MVector K2;
31
32         MVector K3;
33
34         MVector K4;
35
36         // Constructor: initialise the vectors to hold two entries.
37         AdvectionElement(): Left_neighbour_pt(nullptr), Right_neighbour_pt(nullptr), X(2),
            U(2), Utemp(2), K1(2), K2(2), K3(2), K4(2)
38     {}
39
40     virtual ~AdvectionElement() = default;
41
42     // Return the value of the coordinate at local coordinate s using
43     // equation (1.2)
44     double interpolated_x(double s)
45     {return 0.5*(X[0]+X[1]+s*(X[1]-X[0]));}
46
47     // Return the value of the unknown at local coordinate s using
48     // equation (1.4)
49     double interpolated_u(double s)
50     {return 0.5*(Utemp[0]+Utemp[1]+s*(Utemp[1]-Utemp[0]));}
51
52     // Calculate the flux
53     virtual double flux(double u)
54     {
55         return u;
56     }
57
58     // flux derivative
59     virtual double max_flux_derivative(double a, double b)
60     {
61         return 1.0;
62     }
63
64     // Calculate the integral of the flux function over the element

```

```

65 // using the two-point Gauss rule
66 double integrate_flux()
67 {
68     return flux(interpolated_u(-1/std::sqrt(3)))+flux(interpolated_u(1/std::sqrt(3)));
69 }
70
71 // numerical flux
72 virtual double h(double a, double b)
73 {
74     return 0.5*(flux(a)+flux(b)-max_flux_derivative(a, b)*(b-a));
75 }
76
77 // helpful for rk4
78 double F_0_plus_h_0()
79 {
80     return -0.5*integrate_flux()+h((*Left_neighbour_pt).Utemp[1],Utemp[0]);
81 }
82
83 // helpful for rk4
84 double F_1_plus_h_1()
85 {
86     return 0.5*integrate_flux()-h(Utemp[1],(*Right_neighbour_pt).Utemp[0]);
87 }
88
89 // udot
90 MVector udot(double minbound, double maxbound, int N)
91 {
92     double stepsize=(maxbound-minbound)/N;
93     MVector UD(2);
94     UD[0]=(2.0/stepsize)*(2*(F_0_plus_h_0())-(F_1_plus_h_1()));
95     UD[1]=(2.0/stepsize)*(-(F_0_plus_h_0())+2*(F_1_plus_h_1()));
96     return UD;
97 }
98
99 // updating function with Eiger 1
100 void timestep(double dt, double minbound, double maxbound, int N)
101 {
102     U=Utemp+dt*udot(minbound, maxbound, N);
103 }
104
105 // updating function with RK4
106 void timestep_RK4(double dt)
107 {
108     U=U+(dt/6.0)*(K1+2*K2+2*K3+K4);
109 }
110
111 // minmod function
112 double minmod(double sl, double sc, double sr, double a)
113 {
114     if (sl > 0.0 && sc > 0.0 && sr > 0.0)
115     {
116         return std::min({a*sl, sc, a*sr});
117     }
118     if (sl < 0.0 && sc < 0.0 && sr < 0.0)
119     {
120         return std::max({a*sl, sc, a*sr});
121     }
122     return 0.0;
123 }
124
125 void minmod_limiter(double a)

```



```

126     {
127         double ubar = 0.5 * (Utemp[0] + Utemp[1]);
128         double ubarL = 0.5 * ((*Left_neighbour_pt).Utemp[0] +
129                               (*Left_neighbour_pt).Utemp[1]);
130         double ubarR = 0.5 * ((*Right_neighbour_pt).Utemp[0] +
131                               (*Right_neighbour_pt).Utemp[1]);
132         double sl = ubar - ubarL;
133         double sr = ubarR - ubar;
134         double sc = Utemp[1] - Utemp[0];
135         double s_new = minmod(sl, sc, sr, a);
136         Utemp[0] = ubar - 0.5 * s_new;
137         Utemp[1] = ubar + 0.5 * s_new;
138     }
139
140
141 }; //End of the class definition
142
143 // Burger's element class
144 class BurgersElement: public AdvectionElement
145 {
146     public:
147         // Calculate the flux
148         virtual double flux(double u)
149         {
150             return 0.5*(u*u);
151         }
152
153         // flux derivative
154         virtual double max_flux_derivative(double a, double b)
155         {
156             return std::max(std::abs(a),std::abs(b));
157         }
158 };
159
160 // galerkin solver with euler 1
161 void Galerkin_Solver(std::vector<AdvectionElement*> &elements, double dt, double minbound,
162                     double maxbound)
163 {
164     for (int i=0; i<elements.size(); i++)
165     {
166         elements[i]->Utemp=elements[i]->U;
167     }
168     for (int i=0; i<elements.size(); i++)
169     {
170         elements[i]->timestep(dt, minbound, maxbound, elements.size());
171     }
172 }
173
174 // galerkin solver with euler 1 and minmod
175 void Galerkin_Solver_minmod(std::vector<AdvectionElement*> &elements, double dt, double
176                             minbound, double maxbound, double a)
177 {
178     for (int i=0; i<elements.size(); i++)
179     {
180         elements[i]->Utemp=elements[i]->U;
181     }
182     for (int i = 0; i < elements.size(); i++)
183     {
184         elements[i]->minmod_limiter(a);
185     }
186     for (int i=0; i<elements.size(); i++)

```

```

185     {
186         elements[i]->timestep(dt, minbound, maxbound, elements.size());
187     }
188 }
189
190 // implements 1 Galerkin Iteration with RK4
191 void Galerkin_Solver_RK4(std::vector<AdvectionElement*> &elements, double dt, double
    minbound, double maxbound)
192 {
193     for (int i=0; i<elements.size(); i++)
194     {
195         elements[i]->Utemp=elements[i]->U;
196     }
197     for (int i=0; i<elements.size(); i++)
198     {
199         elements[i]->K1=elements[i]->udot(minbound, maxbound, elements.size());
200     }
201     for (int i=0; i<elements.size(); i++)
202     {
203         elements[i]->Utemp=elements[i]->U+(dt/2.0)*elements[i]->K1;
204     }
205     for (int i=0; i<elements.size(); i++)
206     {
207         elements[i]->K2=elements[i]->udot(minbound, maxbound, elements.size());
208     }
209     for (int i=0; i<elements.size(); i++)
210     {
211         elements[i]->Utemp=elements[i]->U+(dt/2.0)*elements[i]->K2;
212     }
213     for (int i=0; i<elements.size(); i++)
214     {
215         elements[i]->K3=elements[i]->udot(minbound, maxbound, elements.size());
216     }
217     for (int i=0; i<elements.size(); i++)
218     {
219         elements[i]->Utemp=elements[i]->U+dt*elements[i]->K3;
220     }
221     for (int i=0; i<elements.size(); i++)
222     {
223         elements[i]->K4=elements[i]->udot(minbound, maxbound, elements.size());
224     }
225     for (int i=0; i<elements.size(); i++)
226     {
227         elements[i]->timestep_RK4(dt);
228     }
229 }
230
231 // implements 1 Galerkin Iteration with RK4 and minmod limiter
232 void Galerkin_Solver_RK4_minmod(std::vector<AdvectionElement*> &elements,
233     double dt, double minbound, double maxbound, double a)
234 {
235     // ---- Stage 1 ----
236     for (int i = 0; i < elements.size(); i++)
237     {
238         elements[i]->Utemp = elements[i]->U;
239     }
240
241     for (int i = 0; i < elements.size(); i++)
242     {
243         elements[i]->minmod_limiter(a);
244     }

```

```

245
246     for (int i = 0; i < elements.size(); i++)
247     {
248         elements[i]->K1 = elements[i]->udot(minbound, maxbound, elements.size());
249     }
250
251     // ---- Stage 2 ----
252     for (int i = 0; i < elements.size(); i++)
253     {
254         elements[i]->Utemp = elements[i]->U + (dt/2.0) * elements[i]->K1;
255     }
256
257     for (int i = 0; i < elements.size(); i++)
258     {
259         elements[i]->minmod_limiter(a);
260     }
261
262     for (int i = 0; i < elements.size(); i++)
263     {
264         elements[i]->K2 = elements[i]->udot(minbound, maxbound, elements.size());
265     }
266
267     // ---- Stage 3 ----
268     for (int i = 0; i < elements.size(); i++)
269     {
270         elements[i]->Utemp = elements[i]->U + (dt/2.0) * elements[i]->K2;
271     }
272
273     for (int i = 0; i < elements.size(); i++)
274     {
275         elements[i]->minmod_limiter(a);
276     }
277
278     for (int i = 0; i < elements.size(); i++)
279     {
280         elements[i]->K3 = elements[i]->udot(minbound, maxbound, elements.size());
281     }
282
283     // ---- Stage 4 ----
284     for (int i = 0; i < elements.size(); i++)
285     {
286         elements[i]->Utemp = elements[i]->U + dt * elements[i]->K3;
287     }
288
289     for (int i = 0; i < elements.size(); i++)
290     {
291         elements[i]->minmod_limiter(a);
292     }
293
294     for (int i = 0; i < elements.size(); i++)
295     {
296         elements[i]->K4 = elements[i]->udot(minbound, maxbound, elements.size());
297     }
298
299     // ---- Final RK4 update ----
300     for (int i = 0; i < elements.size(); i++)
301     {
302         elements[i]->timestep_RK4(dt);
303     }
304 }
305

```

```

306 // sin mesh constructor
307 void sinconstructor(std::vector<AdvectionElement*> &elements, double minbound, double
    maxbound)
308 {
309     double h=(2.0*M_PI)/elements.size();
310     for (int i=0; i<elements.size(); i++)
311     {
312         elements[i]->X[0] = i*h ;
313         elements[i]->X[1] = (i+1)*h;
314         elements[i]->U[0] = 1.5 + sin(elements[i]->X[0]);
315         elements[i]->U[1] = 1.5 + sin(elements[i]->X[1]);
316         if (i > 0)
317             elements[i]->Left_neighbour_pt = elements[i-1];
318         else
319             elements[i]->Left_neighbour_pt = elements[elements.size()-1]; // boundary
320
321         if (i < elements.size()-1)
322             elements[i]->Right_neighbour_pt = elements[i+1];
323         else
324             elements[i]->Right_neighbour_pt = elements[0]; // boundary
325     }
326 }
327
328 // square wave constructor
329 void squareconstructor(std::vector<AdvectionElement*> &elements, double minbound, double
    maxbound)
330 {
331     double h=2.0/elements.size();
332     for (int i=0; i<elements.size(); i++)
333     {
334         elements[i]->X[0] = i*h ;
335         elements[i]->X[1] = (i+1)*h;
336         if(i<=elements.size()/2){elements[i]->U[0] = 1.0;}
337         else {elements[i]->U[0] = 0.0;}
338         if(i+1<=elements.size()/2){elements[i]->U[1] = 1.0;}
339         else {elements[i]->U[1] = 0.0;}
340         if (i > 0)
341             elements[i]->Left_neighbour_pt = elements[i-1];
342         else
343             elements[i]->Left_neighbour_pt = elements[elements.size()-1]; // boundary
344
345         if (i < elements.size()-1)
346             elements[i]->Right_neighbour_pt = elements[i+1];
347         else
348             elements[i]->Right_neighbour_pt = elements[0]; // boundary
349     }
350 }
351
352 // plots the progression of an initialisation
353 void progression_plotter(const std::string& filename, std::vector<AdvectionElement*> &
    elements, double T, double dt, double minbound, double maxbound)
354 {
355     std::ofstream demofile;
356     demofile.open(filename);
357     if (!demofile)
358     {
359         std::cout<<"unable to open file";
360         return;
361     }
362     int endtime=ceil(T/dt)+1;
363     for (int i=0; i<endtime; i++)

```

```

364     {
365         for (int j=0; j<elements.size(); j++)
366         {
367             demofile << elements[j]->interpolated_u(0.0) << " ";
368         }
369         Galerkin_Solver(elements, dt, minbound, maxbound);
370     }
371     demofile.close();
372 }
373
374 // residual computation for sin
375 double sin_adv_error(std::vector<AdvectionElement*> &elements, double T, double minbound,
376                     double maxbound)
377 {
378     double sum=0.0;
379     double h=(maxbound-minbound)/elements.size();
380     for(int i=0; i<elements.size(); i++)
381     {
382         sum+=pow(elements[i]->interpolated_u(0.0)-(1.5+sin(elements[i]->interpolated_x(0.0)
383             -T)),2);
384     }
385     return std::sqrt(h*sum);
386 }
387
388 int main()
389 {
390     /* testing Advection element class initilisation
391     {
392         int N = 7;
393         double h=(2.0*M_PI)/N;
394         std::vector<AdvectionElement*> elements(N);
395         for (int i = 0; i < N; ++i)
396         {
397             elements[i] = new AdvectionElement();
398         }
399         for (int i=0; i<N; i++)
400         {
401             elements[i]->X[0] = i*h ;
402             elements[i]->X[1] = (i+1)*h;
403             elements[i]->U[0] = 1.5 + sin(elements[i]->X[0]);
404             elements[i]->U[1] = 1.5 + sin(elements[i]->X[1]);
405             elements[i]->Utemp=elements[i]->U;
406             if (i > 0)
407                 elements[i]->Left_neighbour_pt = elements[i-1];
408             else
409                 elements[i]->Left_neighbour_pt = elements[N-1]; // boundary
410
411             if (i < N-1)
412                 elements[i]->Right_neighbour_pt = elements[i+1];
413             else
414                 elements[i]->Right_neighbour_pt = elements[0]; // boundary
415
416             // write code to fill out other member data for the i'th element
417             // here
418         }
419         for (int i=0; i<elements.size(); i++)
420         {
421             std::cout << elements[i]->interpolated_x(0.0) << "," << elements[i]->
422                 interpolated_u(0.0) << std::endl;
423         }
424     }
425 }

```

```

422  */
423
424  /* advection with sin
425  {
426      int N=100;
427      double dt=0.001;
428      double t=0.0;
429      double minbound=0.0;
430      double maxbound=2*M_PI;
431      std::vector<AdvectionElement*> elements(N);
432      for (int i = 0; i < N; ++i)
433      {
434          elements[i] = new AdvectionElement();
435      }
436      sinconstructor(elements,minbound,maxbound);
437      for (int i=0; i<1000; i++)
438      {
439          Galerkin_Solver(elements, dt, minbound, maxbound);
440      }
441      for (int i=0; i<elements.size(); i++)
442      {
443          std::cout << elements[i]->interpolated_u(0.0) << std::endl;
444      }
445  }
446  */
447
448  /* progression plotter with sin
449  {
450      int N=100;
451      double dt=0.001;
452      double minbound=0.0;
453      double maxbound=2*M_PI;
454      std::vector<AdvectionElement*> elements(N);
455      for (int i = 0; i < N; ++i)
456      {
457          elements[i] = new AdvectionElement();
458      }
459      sinconstructor(elements,minbound,maxbound);
460      progression_plotter("progression_of_sin_adv.txt", elements, 1.0, dt, minbound,
461                          maxbound);
462  }
463  */
464
465  /* error dependence on timestep
466  {
467      int N=500;
468      double minbound=0.0;
469      double maxbound=2*M_PI;
470      std::vector<AdvectionElement*> elements(N);
471      for (int i = 0; i < N; ++i)
472      {
473          elements[i] = new AdvectionElement();
474      }
475      sinconstructor(elements,minbound,maxbound);
476      for(int i=3750; i>3500; i--)
477      {
478          double dt=1.0/i;
479          for (int j=0; j<i; j++)
480          {
481              Galerkin_Solver(elements, dt, minbound, maxbound);

```

```

482         std::cout<<sin_adv_error(elements, 1.0, minbound, maxbound)<<std::endl;
483         sinconstructor(elements,minbound,maxbound);
484     }
485 }
486 */
487
488 /* error dependence on h
489 {
490     double dt=1.0/10;
491     double minbound=0.0;
492     double maxbound=2*M_PI;
493     for(int i=60; i>59; i--)
494     {
495         int N=i;
496         std::vector<AdvectionElement*> elements(N);
497         for (int i = 0; i < N; ++i)
498         {
499             elements[i] = new AdvectionElement();
500         }
501         sinconstructor(elements,minbound,maxbound);
502         for (int j=0; j<10; j++)
503         {
504             Galerkin_Solver(elements, dt, minbound, maxbound);
505         }
506         std::cout<<sin_adv_error(elements, 1.0, minbound, maxbound)<<std::endl;
507     }
508 }
509 */
510
511 /* error depends on dt and h
512 {
513     std::ofstream demofile;
514     demofile.open("Error_surface.txt");
515     if (!demofile)
516     {
517         std::cout<<"unable to open file";
518         return 1;
519     }
520     double minbound=0.0;
521     double maxbound=2*M_PI;
522     for(int i=6000; i>=1200; i=i-600)
523     {
524         int N=i;
525         std::vector<AdvectionElement*> elements(N);
526         for (int i = 0; i < N; ++i)
527         {
528             elements[i] = new AdvectionElement();
529         }
530         sinconstructor(elements,minbound,maxbound);
531         for (int j=1000; j>=200; j=j-100)
532         {
533             double dt=1.0/j;
534             for (int k=0; k<j; k++)
535             {
536                 Galerkin_Solver(elements, dt, minbound, maxbound);
537             }
538             double a=sin_adv_error(elements, 1.0, minbound, maxbound);
539             if (a>0.4) {demofile<<0.4<<" ";}
540             else {demofile<<a<<" ";}
541             sinconstructor(elements,minbound,maxbound);
542         }

```

```

543     for (int j=100; j>=30; j=j-10)
544     {
545         double dt=1.0/j;
546         for (int k=0; k<j; k++)
547         {
548             Galerkin_Solver(elements, dt, minbound, maxbound);
549         }
550         double a=sin_adv_error(elements, 1.0, minbound, maxbound);
551         if (a>0.4) {demofile<<0.4<<" "};
552         else {demofile<<a<<" "};
553         sinconstructor(elements,minbound,maxbound);
554     }
555     for (int j=20; j>=10; j--)
556     {
557         double dt=1.0/j;
558         for (int k=0; k<j; k++)
559         {
560             Galerkin_Solver(elements, dt, minbound, maxbound);
561         }
562         double a=sin_adv_error(elements, 1.0, minbound, maxbound);
563         if (a>0.4) {demofile<<0.4<<" "};
564         else {demofile<<a<<" "};
565         sinconstructor(elements,minbound,maxbound);
566     }
567 }
568 for(int i=600; i>=180; i=i-60)
569 {
570     int N=i;
571     std::vector<AdvectionElement*> elements(N);
572     for (int i = 0; i < N; ++i)
573     {
574         elements[i] = new AdvectionElement();
575     }
576     sinconstructor(elements,minbound,maxbound);
577     for (int j=1000; j>=200; j=j-100)
578     {
579         double dt=1.0/j;
580         for (int k=0; k<j; k++)
581         {
582             Galerkin_Solver(elements, dt, minbound, maxbound);
583         }
584         double a=sin_adv_error(elements, 1.0, minbound, maxbound);
585         if (a>0.4) {demofile<<0.4<<" "};
586         else {demofile<<a<<" "};
587         sinconstructor(elements,minbound,maxbound);
588     }
589     for (int j=100; j>=30; j=j-10)
590     {
591         double dt=1.0/j;
592         for (int k=0; k<j; k++)
593         {
594             Galerkin_Solver(elements, dt, minbound, maxbound);
595         }
596         double a=sin_adv_error(elements, 1.0, minbound, maxbound);
597         if (a>0.4) {demofile<<0.4<<" "};
598         else {demofile<<a<<" "};
599         sinconstructor(elements,minbound,maxbound);
600     }
601     for (int j=20; j>=10; j--)
602     {
603         double dt=1.0/j;

```



```

604         for (int k=0; k<j; k++)
605         {
606             Galerkin_Solver(elements, dt, minbound, maxbound);
607         }
608         double a=sin_adv_error(elements, 1.0, minbound, maxbound);
609         if (a>0.4) {demofile<<0.4<<" "};
610         else {demofile<<a<<" "};
611         sinconstructor(elements,minbound,maxbound);
612     }
613 }
614 for(int i=120; i>=60; i=i-6)
615 {
616     int N=i;
617     std::vector<AdvectionElement*> elements(N);
618     for (int i = 0; i < N; ++i)
619     {
620         elements[i] = new AdvectionElement();
621     }
622     sinconstructor(elements,minbound,maxbound);
623     for (int j=1000; j>=200; j=j-100)
624     {
625         double dt=1.0/j;
626         for (int k=0; k<j; k++)
627         {
628             Galerkin_Solver(elements, dt, minbound, maxbound);
629         }
630         double a=sin_adv_error(elements, 1.0, minbound, maxbound);
631         if (a>0.4) {demofile<<0.4<<" "};
632         else {demofile<<a<<" "};
633         sinconstructor(elements,minbound,maxbound);
634     }
635     for (int j=100; j>=30; j=j-10)
636     {
637         double dt=1.0/j;
638         for (int k=0; k<j; k++)
639         {
640             Galerkin_Solver(elements, dt, minbound, maxbound);
641         }
642         double a=sin_adv_error(elements, 1.0, minbound, maxbound);
643         if (a>0.4) {demofile<<0.4<<" "};
644         else {demofile<<a<<" "};
645         sinconstructor(elements,minbound,maxbound);
646     }
647     for (int j=20; j>=10; j--)
648     {
649         double dt=1.0/j;
650         for (int k=0; k<j; k++)
651         {
652             Galerkin_Solver(elements, dt, minbound, maxbound);
653         }
654         double a=sin_adv_error(elements, 1.0, minbound, maxbound);
655         if (a>0.4) {demofile<<0.4<<" "};
656         else {demofile<<a<<" "};
657         sinconstructor(elements,minbound,maxbound);
658     }
659 }
660 demofile.close();
661 }
662 */
663
664 /* square constructor test

```

```

665     {
666         int N=3;
667         double minbound=0.0;
668         double maxbound=2.0;
669         std::vector<AdvectionElement*> elements(N);
670         for (int i = 0; i < N; ++i)
671         {
672             elements[i] = new AdvectionElement();
673         }
674         squareconstructor(elements,minbound, maxbound);
675         for (int i=0; i<elements.size(); i++)
676         {
677             std::cout << elements[i]->interpolated_x(0.0) << "," << elements[i]->
                interpolated_u(0.0) << std::endl;
678         }
679     }
680     */
681
682     /* advection with square
683     {
684         int N=100;
685         double dt=0.001;
686         double t=0.0;
687         double minbound=0.0;
688         double maxbound=2.0;
689         std::vector<AdvectionElement*> elements(N);
690         for (int i = 0; i < N; ++i)
691         {
692             elements[i] = new AdvectionElement();
693         }
694         squareconstructor(elements,minbound,maxbound);
695         for (int i=0; i<1000; i++)
696         {
697             Galerkin_Solver(elements, dt, minbound, maxbound);
698         }
699         for (int i=0; i<elements.size(); i++)
700         {
701             elements[i]->Utemp=elements[i]->U;
702             std::cout << elements[i]->interpolated_u(0.0) << std::endl;
703         }
704     }
705     */
706
707
708     /* progression with square
709     {
710         int N=100;
711         double dt=0.00001;
712         double minbound=0.0;
713         double maxbound=2.0;
714         std::vector<AdvectionElement*> elements(N);
715         for (int i = 0; i < N; ++i)
716         {
717             elements[i] = new AdvectionElement();
718         }
719         squareconstructor(elements,minbound,maxbound);
720         progression_plotter("progression_of_square_adv.txt", elements, 1.0, dt, minbound,
                maxbound);
721     }
722     */
723

```

```

724  /* advection with square and minmod
725  {
726      int N=100;
727      double dt=0.001;
728      double t=0.0;
729      double minbound=0.0;
730      double maxbound=2.0;
731      std::vector<AdvectionElement*> elements(N);
732      for (int i = 0; i < N; ++i)
733      {
734          elements[i] = new AdvectionElement();
735      }
736      squareconstructor(elements,minbound,maxbound);
737      for (int i=0; i<1000; i++)
738      {
739          Galerkin_Solver_RK4_minmod(elements, dt, minbound, maxbound, 2.0);
740      }
741      for (int i=0; i<elements.size(); i++)
742      {
743          elements[i]->Utemp=elements[i]->U;
744          std::cout << elements[i]->interpolated_u(0.0) << std::endl;
745      }
746  }
747  */
748
749  /* Burgers with sin
750  {
751      int N=100;
752      double dt=0.001;
753      double t=0.0;
754      double minbound=0.0;
755      double maxbound=2*M_PI;
756      std::vector<AdvectionElement*> elements(N);
757      for (int i = 0; i < N; ++i)
758      {
759          elements[i] = new BurgersElement();
760      }
761      sinconstructor(elements,minbound,maxbound);
762      for (int i=0; i<2000; i++)
763      {
764          Galerkin_Solver_RK4_minmod(elements, dt, minbound, maxbound, 2.0);
765      }
766      for (int i=0; i<elements.size(); i++)
767      {
768          elements[i]->Utemp=elements[i]->U;
769          std::cout << elements[i]->interpolated_u(0.0) << std::endl;
770      }
771      for (auto el : elements)
772      {
773          delete el;
774      }
775      elements.clear();
776  }
777  */
778
779  /* burgers with square and minmod
780  {
781      int N=100;
782      double dt=0.001;
783      double t=0.0;
784      double minbound=0.0;

```

```

785     double maxbound=2.0;
786     std::vector<AdvectionElement*> elements(N);
787     for (int i = 0; i < N; ++i)
788     {
789         elements[i] = new BurgersElement();
790     }
791     squareconstructor(elements,minbound,maxbound);
792     for (int i=0; i<2000; i++)
793     {
794         Galerkin_Solver_RK4_minmod(elements, dt, minbound, maxbound, 2.0);
795     }
796     for (int i=0; i<elements.size(); i++)
797     {
798         elements[i]->Utemp=elements[i]->U;
799         std::cout << elements[i]->interpolated_u(0.0) << std::endl;
800     }
801 }
802
803 */
804
805 /* oscillations illustration
806 {
807     int N=11;
808     double dt=0.04;
809     double t=0.0;
810     double minbound=0.0;
811     double maxbound=2.0;
812     std::vector<AdvectionElement*> elements(N);
813     for (int i = 0; i < N; ++i)
814     {
815         elements[i] = new AdvectionElement();
816     }
817     squareconstructor(elements,minbound,maxbound);
818     for (int i=0; i<3; i++)
819     {
820         Galerkin_Solver(elements, dt, minbound, maxbound);
821     }
822     for (int i=3; i<elements.size()-2; i++)
823     {
824         elements[i]->Utemp=elements[i]->U;
825         std::cout <<elements[i]->Utemp[0]<< std::endl;
826         std::cout <<elements[i]->Utemp[1]<< std::endl;
827     }
828 }
829 */
830
831 // limiters illustration (rember to set square constructor back to <= and
832 // galerkin_minmod to not print)
833 {
834     int N=11;
835     double dt=0.04;
836     double t=0.0;
837     double minbound=0.0;
838     double maxbound=2.0;
839     std::vector<AdvectionElement*> elements(N);
840     for (int i = 0; i < N; ++i)
841     {
842         elements[i] = new AdvectionElement();
843     }
844     squareconstructor(elements,minbound,maxbound);
845     for (int i=0; i<4; i++)

```

```

845     {
846         Galerkin_Solver_minmod(elements, dt, minbound, maxbound,1.0);
847     }
848 }
849
850     return 0;
851 }

```

11.2

```

852 #ifndef MVECTOR_H // the 'include guard'
853 #define MVECTOR_H // see C++ Primer Sec. 2.9.2
854
855 #include <vector>
856 #include <iostream>
857 #include <algorithm>
858 #include <cmath>
859 #include <numeric>
860 #include <random>
861 #include <cstdlib>
862 #include "rand_norm.h"
863 #include <ctime>
864
865 // Class that represents a mathematical vector
866 class MVector
867 {
868 public:
869     // constructors
870     MVector() {}
871     explicit MVector(int n) : v(n) {}
872     MVector(int n, double x) : v(n, x) {}
873     MVector(std::initializer_list<double> l) : v(l) {}
874
875     // access element (lvalue) (see example sheet 5, q5.6)
876     double &operator[](int index)
877     {
878         return v[index];
879     }
880
881     // access element (rvalue) (see example sheet 5, q5.7)
882     double operator[](int index) const {
883         return v[index];
884     }
885
886     int size() const { return v.size(); } // number of elements
887
888     // add data to your vector
889     void push_back(double x){v.push_back(x);}
890
891     // infinity norm
892     double LInfNorm() const
893     {
894         double maxAbs = 0;
895         std::size_t s = size();
896         for (int i=0; i<s; i++)
897         {
898             maxAbs = std::max(std::abs(v[i]), maxAbs);
899         }
900         return maxAbs;
901     }

```

```

902
903 // L2 norm
904 double L2Norm() const
905 {
906     double sum = 0;
907     std::size_t s = size();
908     for (int i=0; i<s; i++)
909     {
910         sum+=v[i]*v[i];
911     }
912     return std::sqrt(sum);
913 }
914
915 auto begin() { return v.begin(); }
916
917 auto end() { return v.end(); }
918
919 auto begin() const { return v.begin(); }
920
921 auto end() const { return v.end(); }
922
923 // Threshold
924 void Threshold(int k)
925 {
926     MVector w(v.size(),0);
927     auto itmin = std::min_element(v.begin(), v.end());
928     int indexmin = std::distance(v.begin(), itmin);
929     auto itmax = std::max_element(v.begin(), v.end());
930     int indexmax = std::distance(v.begin(), itmax);
931     for(int i=0;i<k;i++)
932     {
933         if(std::abs(*itmax)>std::abs(*itmin))
934         {
935             w[indexmax]=*itmax;
936             v[indexmax]=0;
937             itmax = std::max_element(v.begin(), v.end());
938             indexmax = std::distance(v.begin(), itmax);
939         }
940         else
941         {
942             w[indexmin]=*itmin;
943             v[indexmin]=0;
944             itmin = std::min_element(v.begin(), v.end());
945             indexmin = std::distance(v.begin(), itmin);
946         }
947     }
948     *this=w;
949 }
950
951 // sparse initialiser
952 void initialise_sparse_normal(int k)
953 {
954     if (k > size())
955     {
956         std::cout<<"invalid argument"<<std::endl;
957         return;
958     }
959     // Step 2: generate indices [0..size()-1]
960     std::vector<int> indices(size());
961     std::iota(indices.begin(), indices.end(), 0);
962     // Step 3: shuffle them

```

```

963         std::mt19937 gen(static_cast<unsigned>(std::time(nullptr)));
964         std::shuffle(indices.begin(), indices.end(), gen);
965         // Step 4: set k random positions using rand_normal()
966         for (int i = 0; i < k; ++i) {
967             v[indices[i]] = rand_normal();
968         }
969     }
970
971 private:
972     std::vector<double> v;
973 };
974
975 //1.2.1
976
977 inline MVector operator*(const double& lhs, const MVector& rhs)
978 {
979     MVector temp = rhs;
980     for (int i=0; i<temp.size(); i++) temp[i]*=lhs;
981     return temp;
982 }
983
984 inline MVector operator*(const MVector& rhs, const double& lhs)
985 {
986     MVector temp = rhs;
987     for (int i=0; i<temp.size(); i++) temp[i]*=lhs;
988     return temp;
989 }
990
991 inline MVector operator/(const MVector& rhs, const double& lhs)
992 {
993     MVector temp = rhs;
994     for (int i=0; i<temp.size(); i++) temp[i]/=lhs;
995     return temp;
996 }
997
998 inline MVector operator+(const MVector& lhs, const MVector& rhs)
999 {
1000     if (lhs.size()!=rhs.size())
1001     {
1002         std::cout<<"incompatible vector size"<<std::endl;
1003         return MVector();
1004     }
1005     MVector temp = rhs;
1006     for (int i=0; i<temp.size(); i++) temp[i]=lhs[i]+rhs[i];
1007     return temp;
1008 }
1009
1010 inline MVector operator-(const MVector& lhs, const MVector& rhs)
1011 {
1012     MVector temp = rhs;
1013     for (int i=0; i<temp.size(); i++) temp[i]=lhs[i]-rhs[i];
1014     return temp;
1015 }
1016
1017 inline double dot(const MVector& lhs, const MVector& rhs)
1018 {
1019     if (lhs.size()!=rhs.size())
1020     {
1021         std::cout<<"incompatible vectors"<<std::endl;
1022         return 1e6;
1023     }

```

```

1024     double sum=0;
1025     for(int i=0;i<lhs.size();i++)
1026     {
1027         sum+=lhs[i]*rhs[i];
1028     }
1029     return sum;
1030 }
1031
1032 std::ostream& operator<<(std::ostream& os, const MVector& v)
1033 {
1034     int n = v.size();
1035     os << "("<<v[0];
1036     for(int i=1; i<n;i++) os<<","<<v[i];
1037     os << ")";
1038     return os;
1039 }
1040
1041 #endif

```

11.3

```

1042 #ifndef MMATRIX_H // the 'include guard'
1043 #define MMATRIX_H
1044
1045 #include <vector>
1046 #include <cstdlib>
1047 #include <cmath>
1048 #include "rand_norm.h"
1049
1050
1051
1052
1053
1054 // Class that represents a mathematical matrix
1055 class MMatrix
1056 {
1057 public:
1058     // constructors
1059     MMatrix() : nRows(0), nCols(0) {}
1060     MMatrix(int n, int m, double x = 0) : nRows(n), nCols(m), A(n * m, x) {}
1061
1062     // set all matrix entries equal to a double
1063     MMatrix &operator=(double x)
1064     {
1065         for (unsigned i = 0; i < nRows * nCols; i++) A[i] = x;
1066         return *this;
1067     }
1068
1069     // access element, indexed by (row, column) [rvalue]
1070     double operator()(int i, int j) const
1071     {
1072         return A[j + i * nCols];
1073     }
1074
1075     // access element, indexed by (row, column) [lvalue]
1076     double &operator()(int i, int j)
1077     {
1078         return A[j + i * nCols];
1079     }
1080

```



```

1081 // size of matrix
1082 int Rows() const { return nRows; }
1083 int Cols() const { return nCols; }
1084
1085 void initialise_normal()
1086 {
1087     for(int i=0; i<nRows;i++)
1088     {
1089         for(int j=0; j<nCols;j++)
1090         {
1091             A[j + i * nCols]=rand_normal()/nRows;
1092         }
1093     }
1094 }
1095
1096 void normalise_columns()
1097 {
1098     for(int j = 0; j < nCols; j++)
1099     {
1100         // compute column norm
1101         double norm = 0.0;
1102         for(int i = 0; i < nRows; i++)
1103         {
1104             double val = A[j + i*nCols];
1105             norm += val * val;
1106         }
1107         norm = std::sqrt(norm);
1108
1109         // avoid division by zero
1110         if(norm > 0.0)
1111         {
1112             for(int i = 0; i < nRows; i++)
1113             {
1114                 A[j + i*nCols] /= norm;
1115             }
1116         }
1117     }
1118 }
1119 }
1120
1121 private:
1122     unsigned int nRows, nCols;
1123     std::vector<double> A;
1124 };
1125
1126 inline MVector operator*(const MMatrix& A, const MVector& x)
1127 {
1128     if (x.size()!=A.Cols())
1129     {
1130         std::cout<<"incompatible operator"<<std::endl;
1131         return {0};
1132     }
1133     MVector v(A.Rows());
1134     for (int i=0; i<A.Rows(); i++)
1135     {
1136         v[i]=0;
1137         for(int j=0; j<A.Cols();j++)
1138         {
1139             v[i]+=A(i,j)*x[j];
1140         }
1141     }

```

```

1142     }
1143     return v;
1144 }
1145
1146 inline MMatrix operator*(const MMatrix& A, const MMatrix& B)
1147 {
1148     if(A.Cols() !=B.Rows())
1149     {
1150         std::cout<<"incompatible matrix operation"<<std::endl;
1151         MMatrix C(0,0,0);
1152         return C;
1153     }
1154     MMatrix D(A.Rows(),B.Cols(),0);
1155     for(int i=0;i<A.Rows();i++)
1156     {
1157         for(int j=0; j<B.Cols();j++)
1158         {
1159             for(int k=0; k<A.Cols();k++)
1160             {
1161                 D(i,j)+=A(i,k)*B(k,j);
1162             }
1163         }
1164     }
1165     return D;
1166 }
1167
1168 inline MMatrix Transpose(const MMatrix& A)
1169 {
1170     MMatrix B(A.Cols(), A.Rows(),0);
1171     for(int i=0; i<A.Cols(); i++)
1172     {
1173         for(int j=0; j<A.Rows(); j++)
1174         {
1175             B(i,j)=A(j,i);
1176         }
1177     }
1178     return B;
1179 }
1180
1181 std::ostream& operator<<(std::ostream& os, const MMatrix& A)
1182 {
1183     os.width(1);
1184     os << "(";
1185     os.width(9);
1186     os.precision(9);
1187     os<<A(0,0);
1188     os<<",";
1189
1190     for(int i=0;i<A.Rows();i++)
1191     {
1192         for(int j=0;j<A.Cols();j++)
1193         {
1194             if (i==0 && j==0) continue;
1195             os.width(10);
1196             os.precision(10);
1197             os<<A(i,j);
1198             if(j<A.Cols()-1) os<<",";
1199         }
1200         if(i<A.Rows()-1) os<<std::endl;
1201     }
1202     os << ")";

```

```
1203     return os;  
1204 }  
1205  
1206 #endif
```
